

第六章 树

张史梁

课程内容

- 6.1 树的概念
- 6.2 树的链式存储
- 6.3 树的顺序存储
- 6.4 K叉树

6.1.1 树的定义

- 树是包括n个结点的有限集合T（n≥1），使得：
  - 一个根结点
  - 除根以外的其它结点被分成m个(m≥0)不相交的集合T1，T2，…，Tm，而且这些集合的每一个又都是树。 树T1，T2，…，Tm称作这个根的子树
- 定义是递归的

逻辑结构

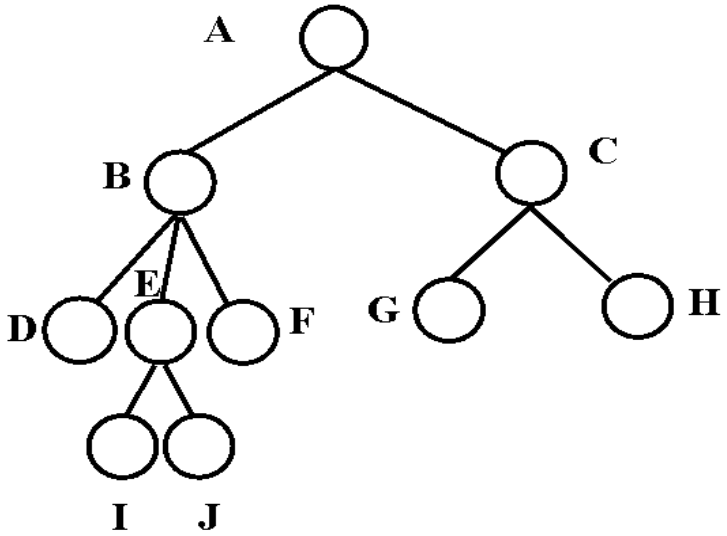
- 包含n个结点的有穷集合 K（n>0），且在K上定义了一个关系N，关系N满足以下条件：
  - 有且仅有一个结点$k_0 \in K$，它对于关系N来说没有前驱。结点k_0称作树的根
  - 除k_0外，K中每个结点对于关系N来说都有且仅有一个前驱
  - 除k_0外，任何结点$k \in K$，存在一结点序列k_0，k_1，…，k_s，使得k_0就是树根，且$k_s=k$，其中有序对$<k_{i-1}, k_i> \in N(1 \leqslant i \leqslant s)$。这样的结点序列称为从根到结点k的一条路径

树形结构的表示法

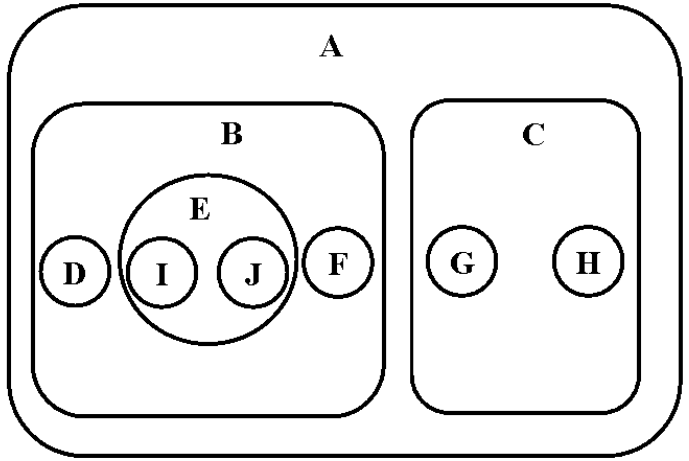
树的逻辑结构

- 结点集合K={A, B, C, D, E, F, G, H, I, J}
- K上的关系N={<A, B>, <A, C>, <B, D>,

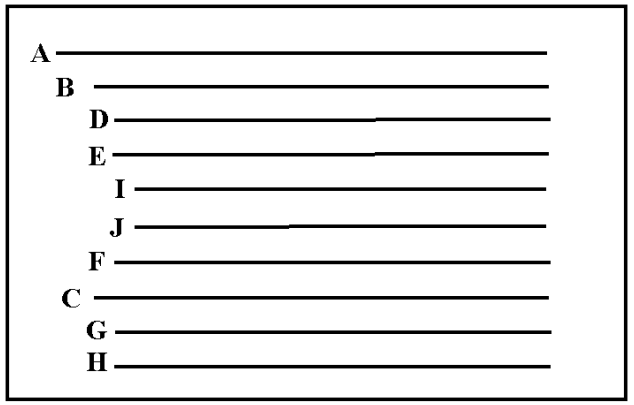
  <B, E>, <B, F>, <C, G>, <C, H>, <E, I>, <E, J>}



(a)树形表示法



(b)文氏图表示法

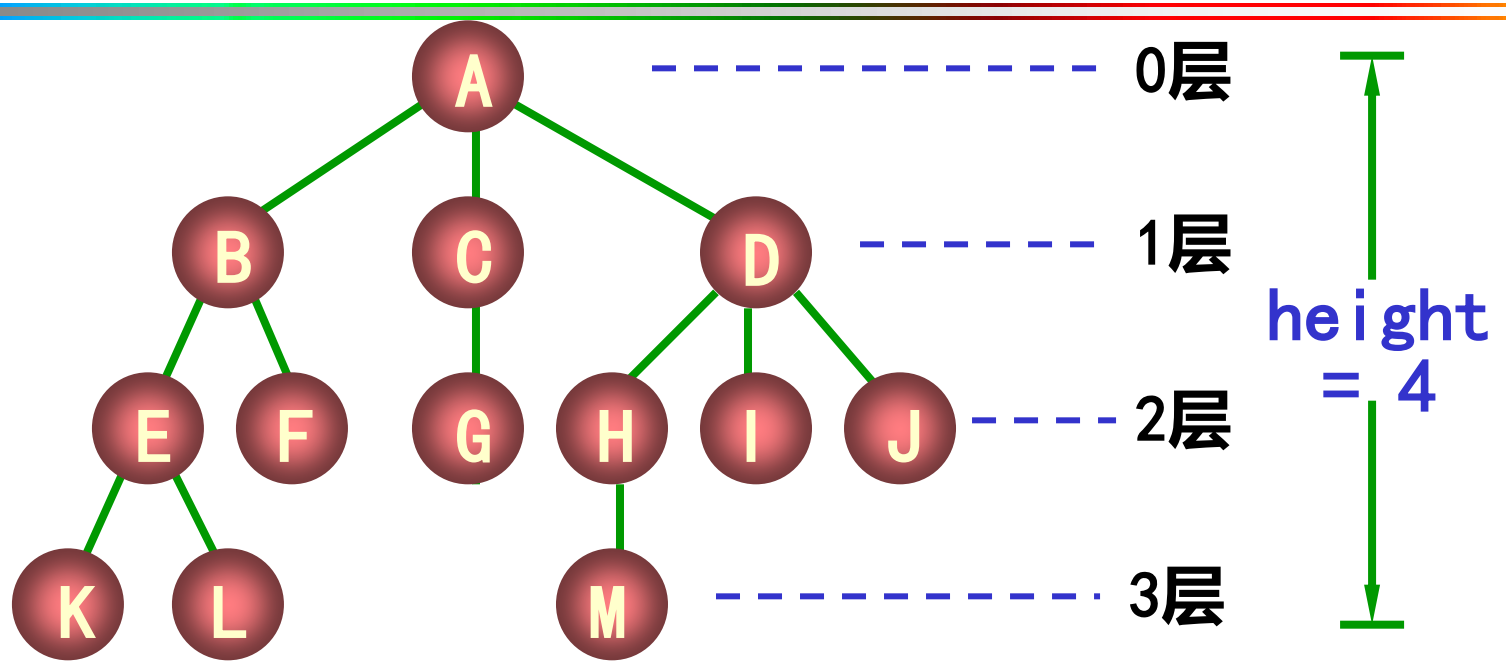


(c)凹入表表示法

(A(B(D)(E(I)(J)(F))(C(G)(H)))

(d)嵌套括号表示法

树的基本术语



结点
结点的度
叶结点
分支结点

子女
双亲
兄弟

祖先
子孙
结点层次

树的深（高）度
树的度
有序树
无序树
森林

树结构中的概念

➢ 有序树

- 把树结点的子结点按从左到右的次序顺序编号

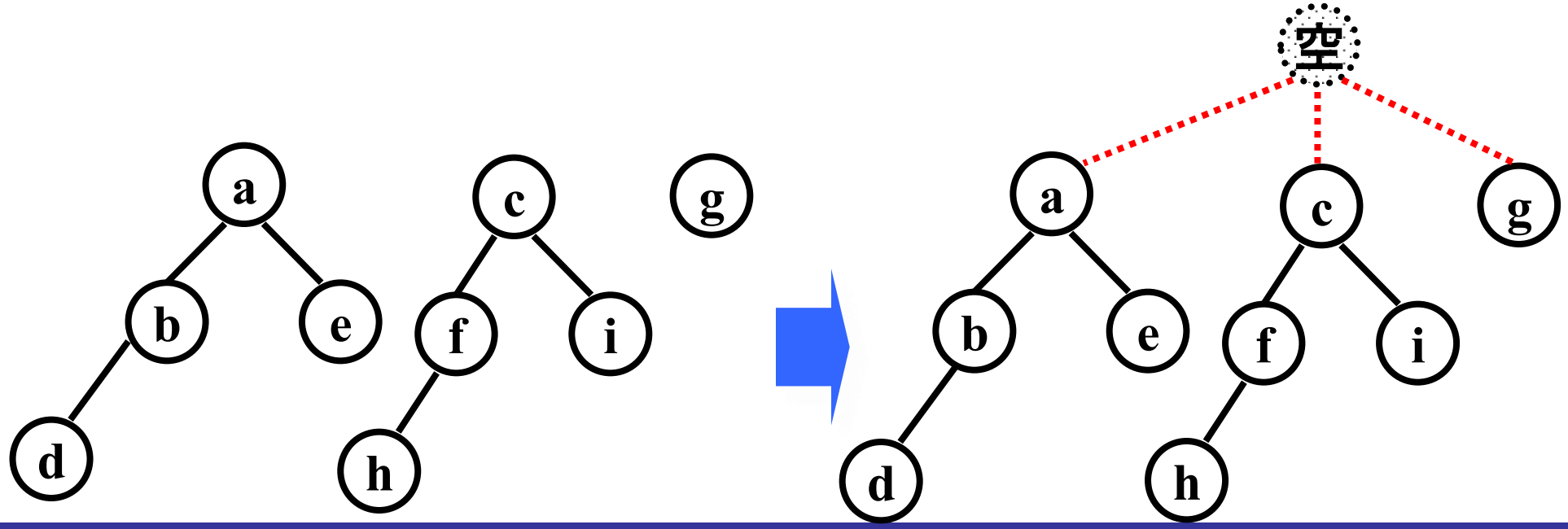
➢ 度为2的有序树是不是二叉树?

- No
- 第一子结点被删除后，第二子结点自然顶替成为第1子结点。
- 度为2并且**严格区分左右**两个子结点的有序树才是二叉树。

	度为 2 的有序树	二叉树
节点的孩子数	每个节点最多 2 个孩子	每个节点最多 2 个孩子
孩子的区分	有顺序，但**不区分左右**	**严格区分左孩子和右孩子**
只有一个孩子时	这个孩子就是“第一个孩子”，没有左右之分	必须明确是左孩子还是右孩子

森林

- 森林(forest)是零棵或多棵不相交的树的集合（通常是有序集合）
- 对于树中的每个结点，其子树组成的集合就是森林；而加入一个结点作为根，森林就可以转化成一棵树了



6.1.2 森林与二叉树的等价转换

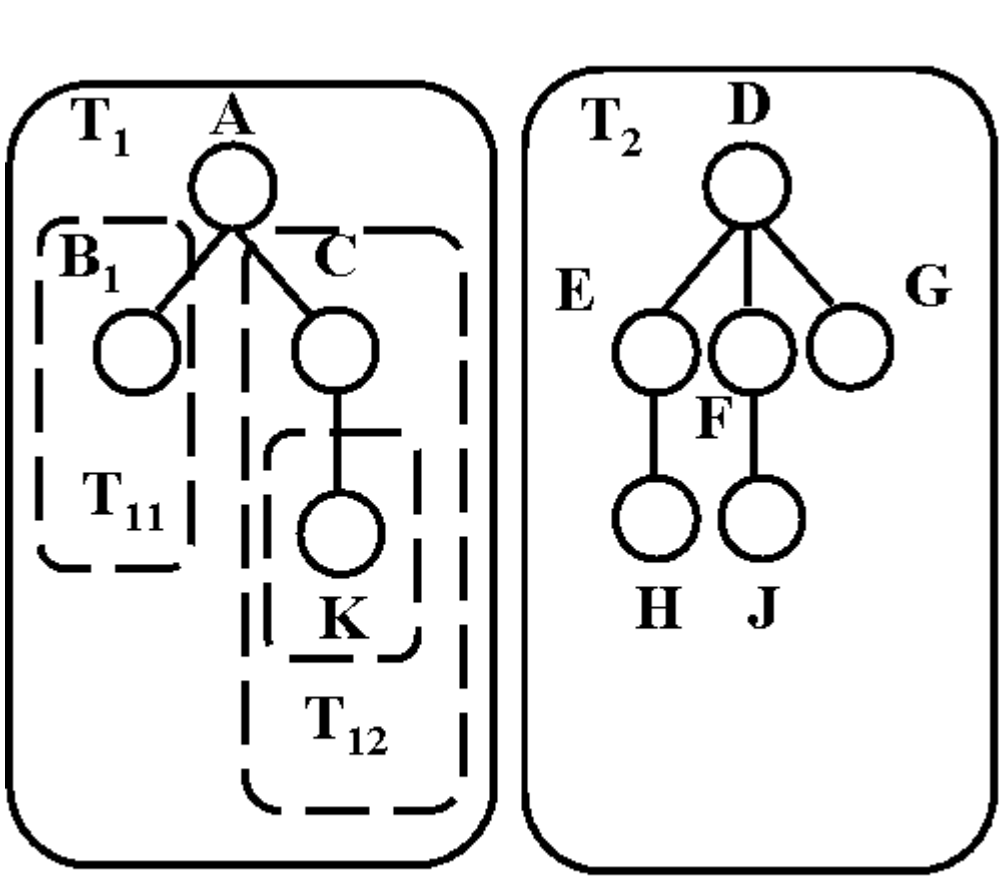
- 树或森林与二叉树一一对应
  - 任何森林都可以用一棵二叉树唯一表达
  - 任何二叉树也都唯一对应到一个森林
- 树所对应的二叉树中
  - 一个结点的左子结点是它在原来树里的第一个子结点
  - 右子结点是它在原来的树里的下一个兄弟

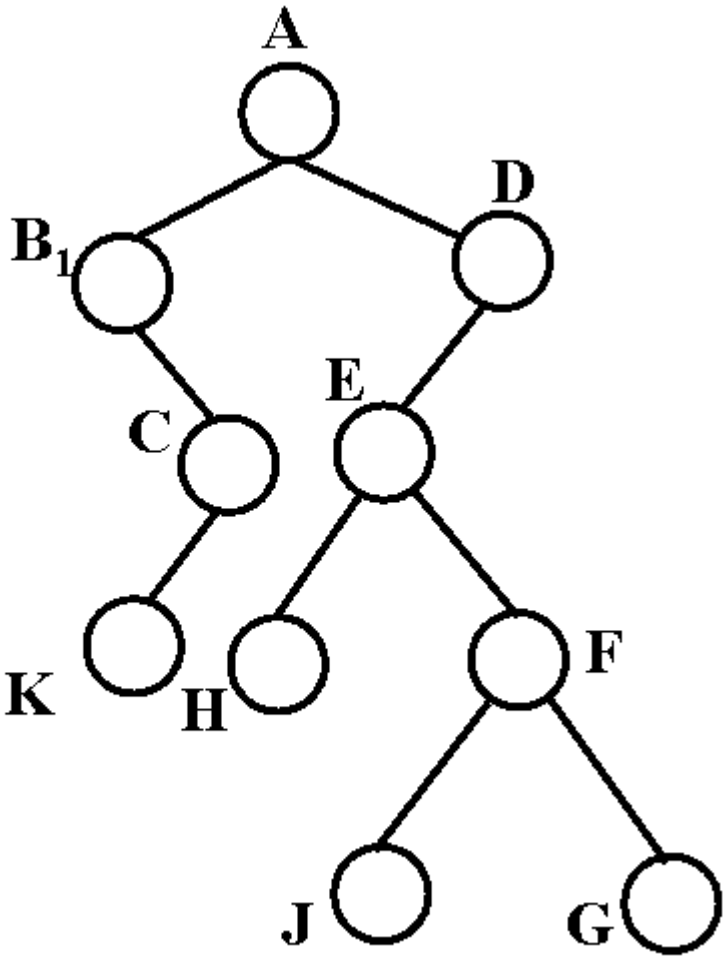
左孩子,右兄弟

森林到二叉树的等价转换

- 把森林F看作树的有序集合，$F=(T_1, T_2, \cdots, T_n)$，对应于F的二叉树B(F)的定义是：
  - 若n=0，则B(F)为空
  - 若n>0，则B(F)的根是T_1的根R_1，B(F)的左子树是$B(T_{11}, T_{12}, \cdots, T_{1m})$，其中$T_{11}, T_{12}, \cdots, T_{1m}$是$R_1$的子树；B(F)的右子树是$B(T_2, \cdots, T_n)$
- 此定义精确地确定了从森林到二叉树的转换

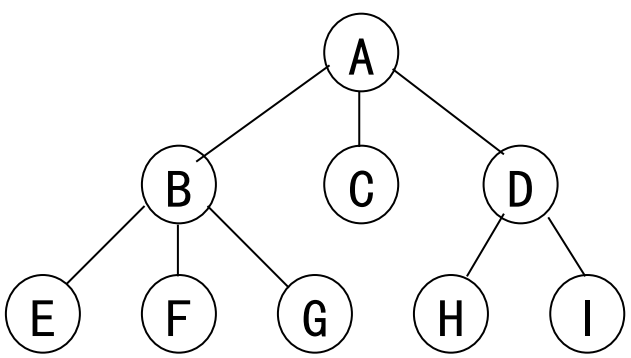
示例1

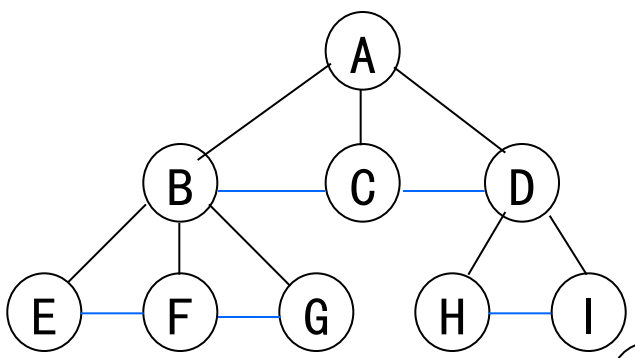


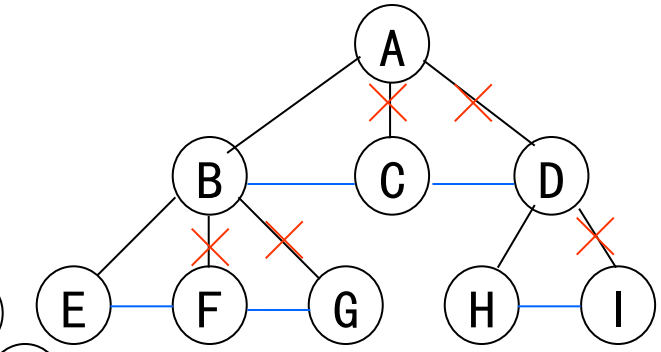


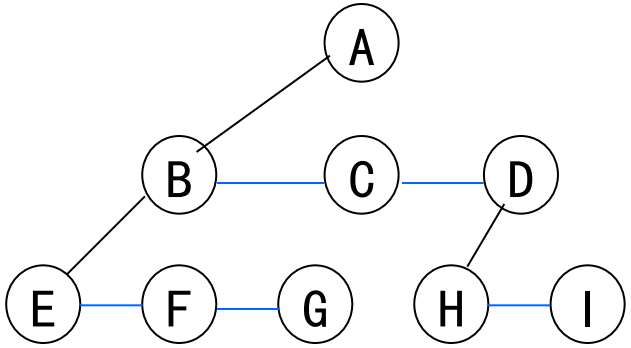
示例2

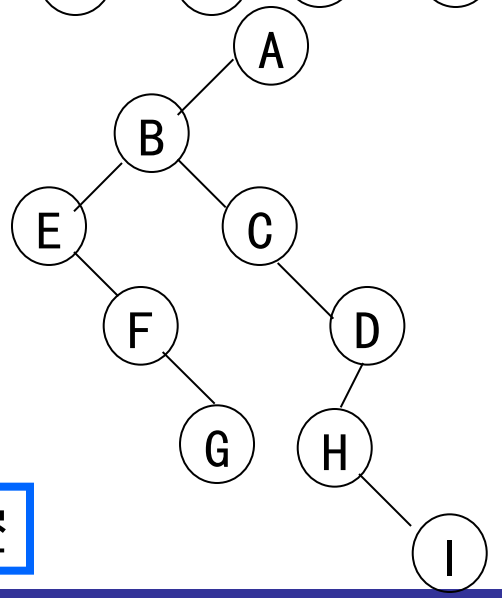
- **加线**：在树中所有相邻的兄弟之间加一连线
- **抹线**：对树中每个结点，除其最左孩子外，抹去该结点与其余孩子间的连线
- **整理**：以树的根结点为轴心，将整树顺时针转45°







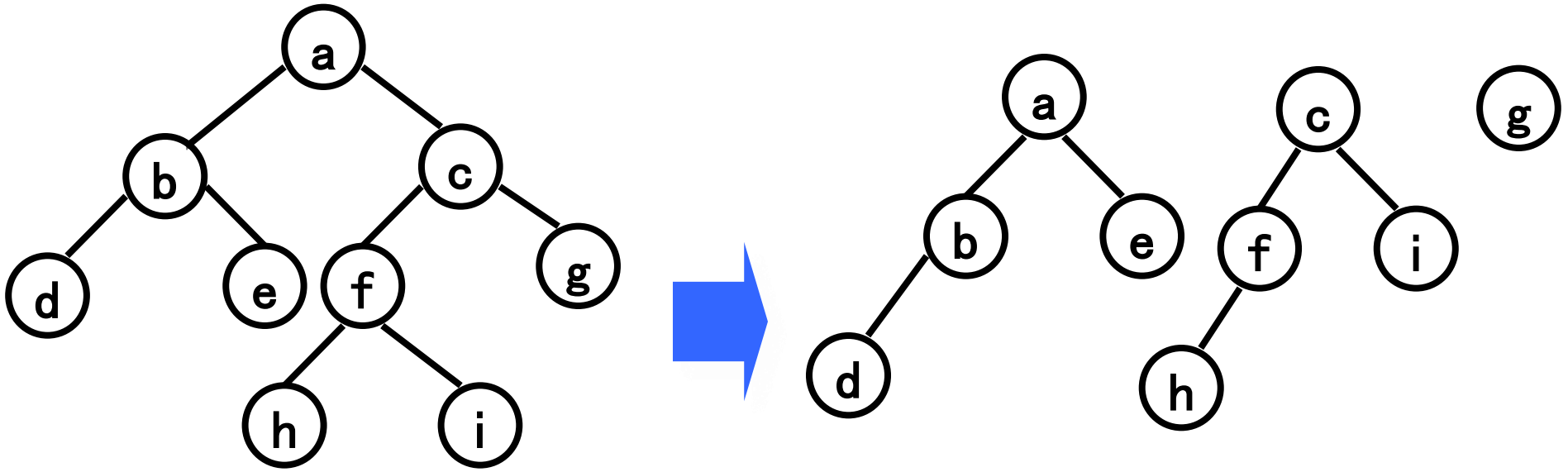




树转换成的二叉树其右子树一定为空

二叉树到森林的等价转换

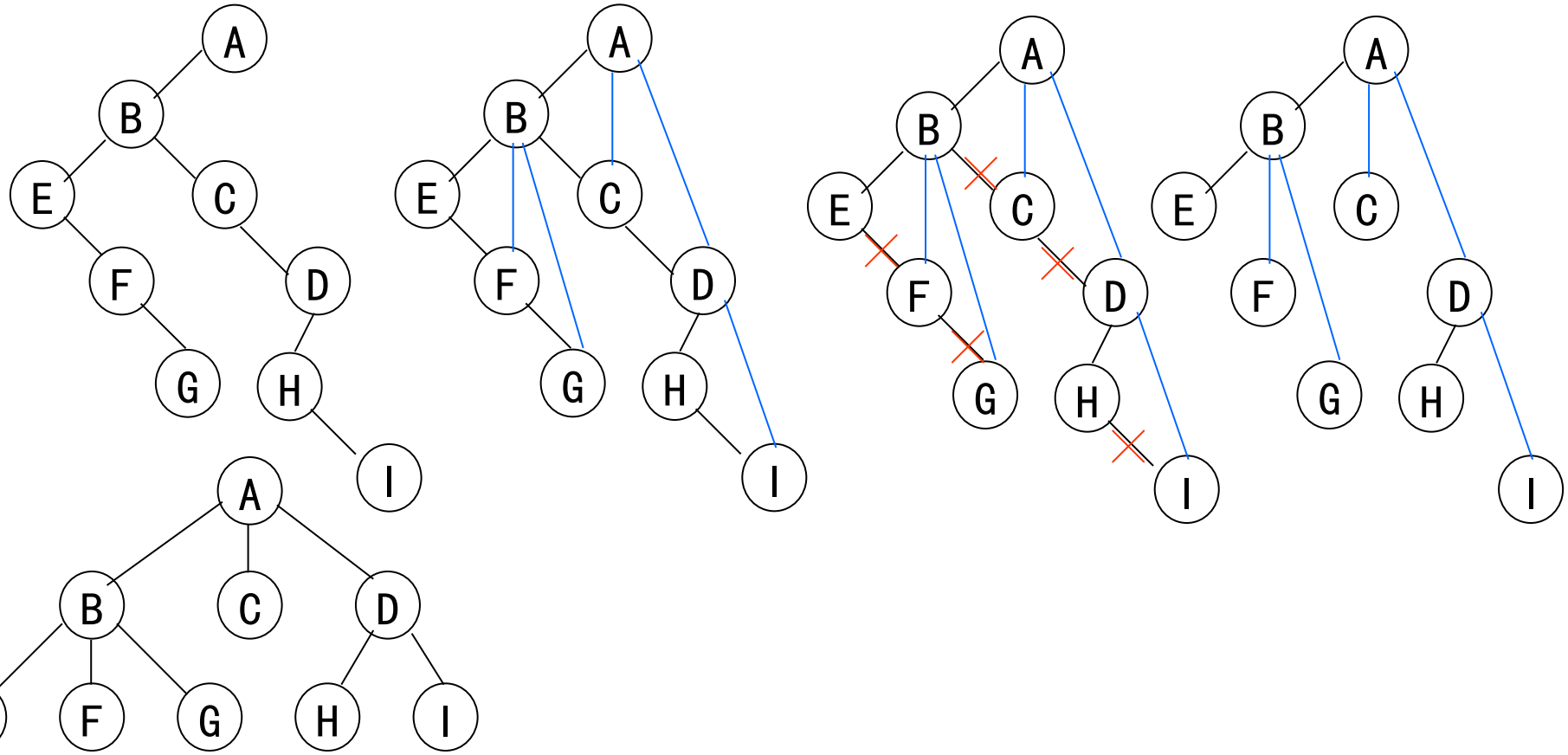
- 设B是一棵二叉树，root是B的根，L和R分别是root的左子树和右子树，则森林F(B)的定义是：
  - 若B为空，则F(B)是空的森林。
  - 若B不为空，则F(B)是一棵树T_1加上森林F(R)，其中树T_1的根为root，root的子树为F(L)



(a)二叉树　　(b)森林

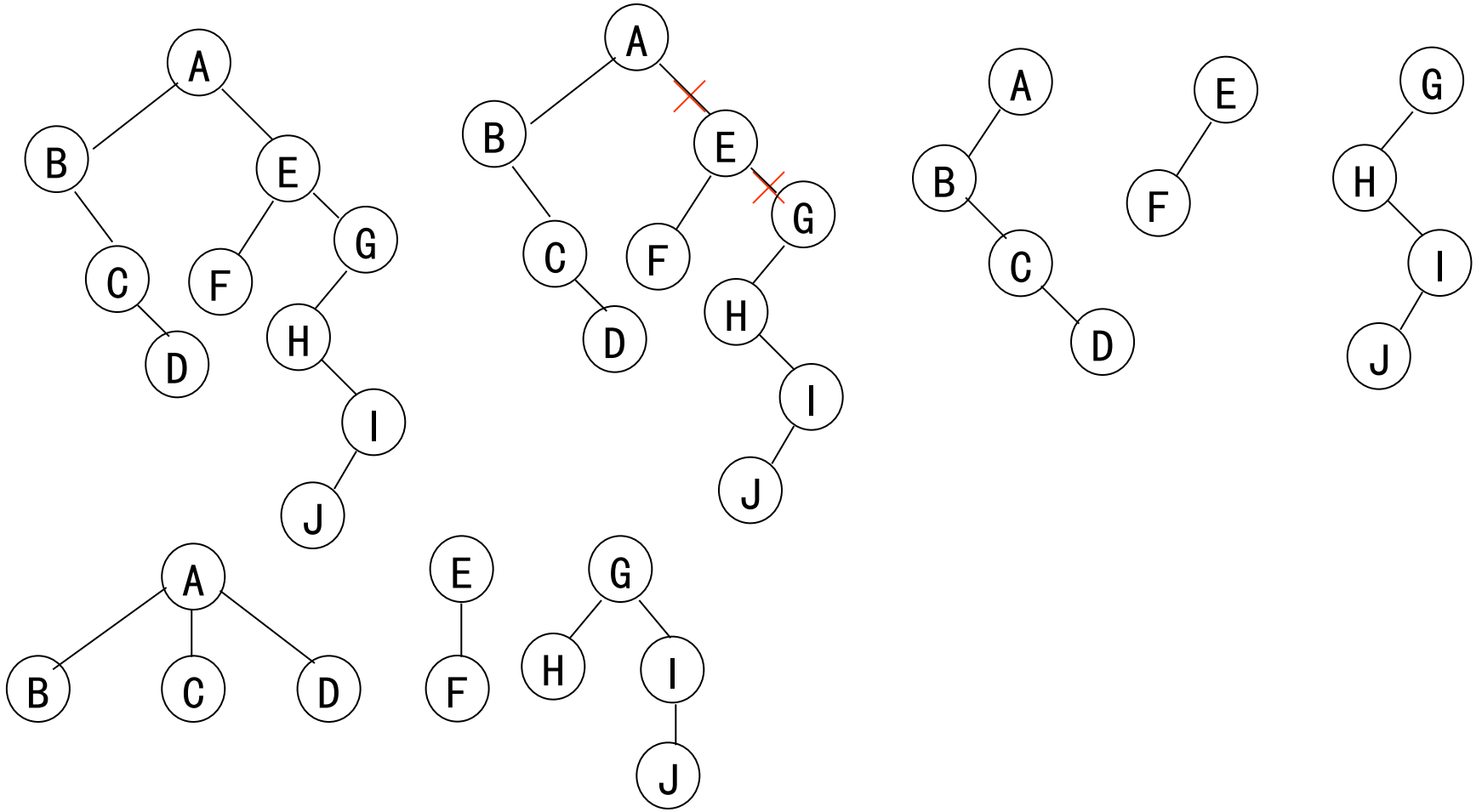
示例1:二叉树转换为树

- **加线**:若p结点是父结点的左孩子,则将p的右孩子,右孩子的右孩子,……沿分支找到的所有右孩子,都与p的双亲用线连起来
- **抹线**:抹掉原二叉树中双亲与右孩子之间的连线
- **调整**:将结点按层次排列,形成树结构



示例2:二叉树转换成森林

- 抹线:将二叉树中根结点与其右孩子连线，及沿右分支搜索到的所有右孩子间连线全部抹掉，使之变成孤立的二叉树
- 还原:将孤立的二叉树还原成树



思考题：

- 利用代码来实现

1. 从树到二叉树的转换；

2. 从二叉树到树的转换

6.1.3 树的抽象数据类型

➢ 节点的抽象数据类型

```
template<class T>
   class TreeNode  {
 public:
  TreeNode(const T&);                          //构造函数
   virtual ~TreeNode(){};                      //析构函数
   bool isLeaf();                              //如果结点是叶，返回true
   T Value();                                  //返回结点的值
   TreeNode<T>* LeftMostChild();               //返回第一个左孩子
   TreeNode<T>* RightSibling();                //返回右兄弟
   void setValue(T&);                          //设置结点的值
    void setChild(TreeNode<T>* pointer);       //设置左子结点
    void setSibling(TreeNode<T>* pointer);     //设置右兄弟
   void InsertFirst(TreeNode<T>* node);        //以第一个左子结点身份插入结点
   void InsertNext(TreeNode<T>* node);         //以右兄弟的身份插入结点
};
```

树的抽象数据类型

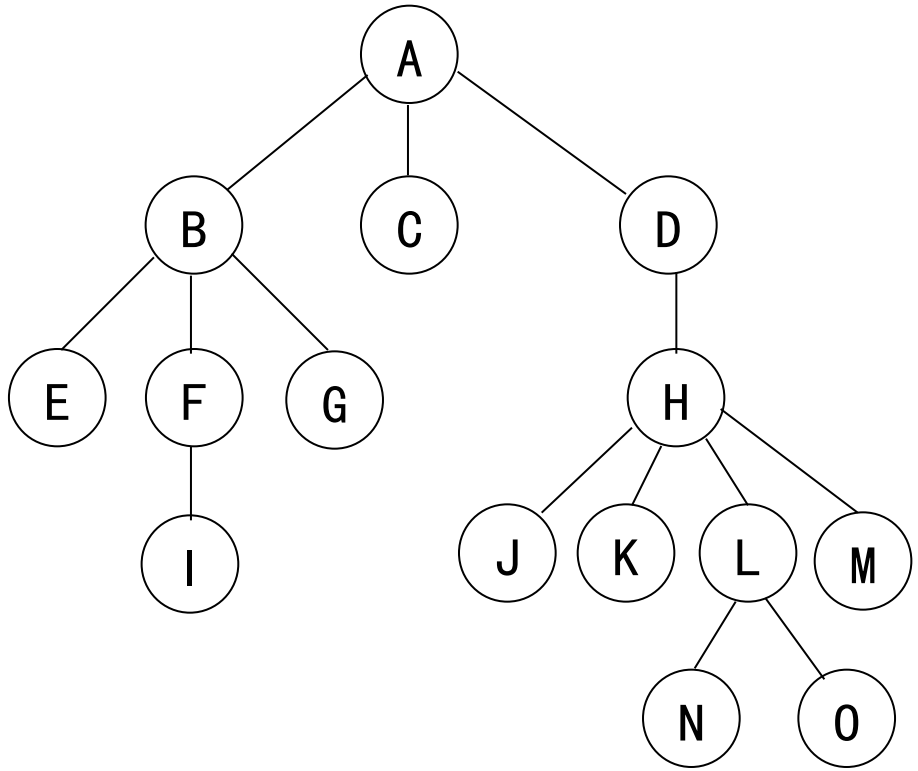
```
template <class T>
  class Tree  {
  public:
      Tree();                          //构造函数
      virtual ~Tree();                 //析构函数
      TreeNode<T>* getRoot();        //返回树中的根结点
      //创建树中的根结点，使根结点元素的值为rootValue
      void CreateRoot(const T& rootValue);
      //判断是否为空树，如果是则返回true
      bool  isEmpty();
```

```
        //返回current结点的父结点
        TreeNode<T>* Parent(TreeNode<T>* current);
        //返回current结点的前一个兄弟结点
        TreeNode<T>* PrevSibling(TreeNode<T>* current);
        //删除以subroot为根的子树的所有结点
        void DeleteSubTree(TreeNode<T>* subroot);
        //先根深度优先周游树
        void RootFirstTraverse(TreeNode<T>* root);
        //后根深度优先周游树
        void RootLastTraverse(TreeNode<T>* root);
        //宽度优先周游树
        void WidthTraverse(TreeNode<T>* root);
};
```

6.1.4 树(森林)的周游

- 按深度的方向周游
  - 先根次序
    - 若树非空，则遍历方法为：
      - 访问根结点
      - 从左到右，依次先根遍历根结点的每一棵子树
  - 后根次序
    - 若树非空，则遍历方法为：
      - 从左到右，依次后根遍历根结点的每一棵子树
      - 访问根结点

无中根次序周游



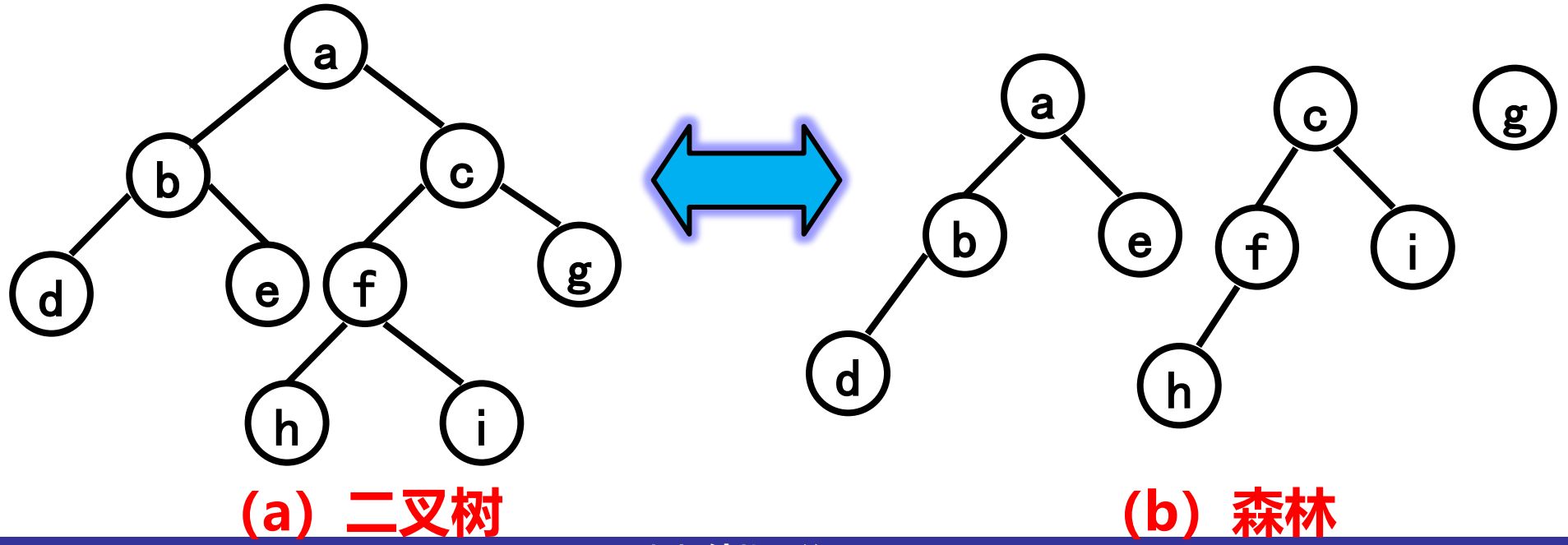
先根遍历：ABEFIGCDHJKLNOM

后根遍历：EIFGBCJKNOLMHDA

层次遍历：ABCDEFGHIJKLMNO

周游性质

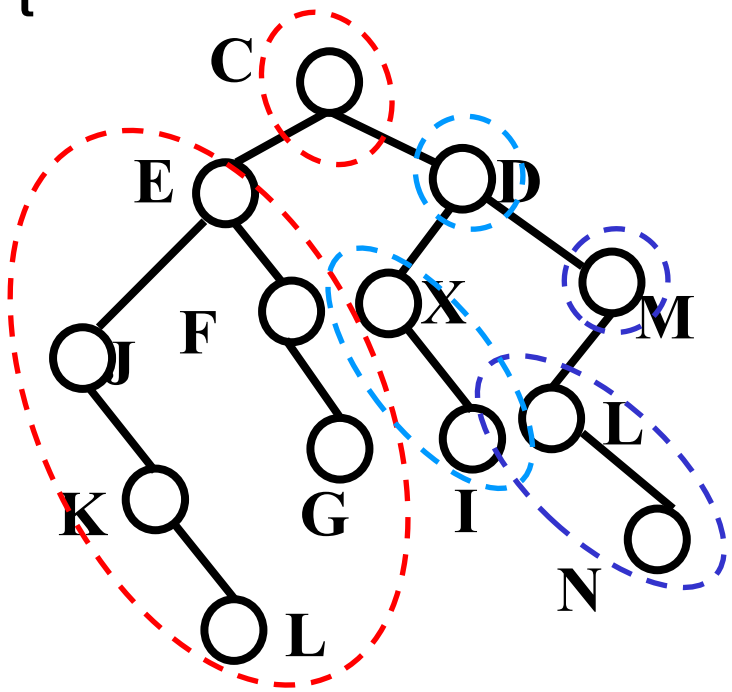
- 按先根次序周游树正好等于对应二叉树的前序周游
  - 周游结果：abdecfhig
- 按后根次序周游树正好等于对应二叉树的中序周游
  - 周游结果：dbeahficg



(a) 二叉树　　(b) 森林

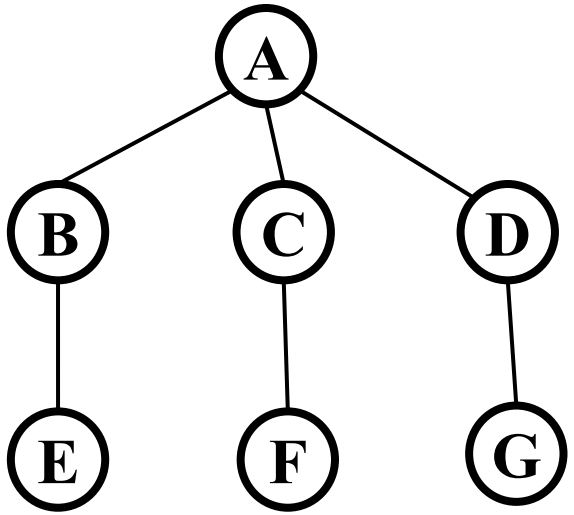
先根深度优先周游树(森林)

```
template <class T>
void Tree<T>::RootFirstTraverse(TreeNode<T>* root){
      while(root!=NULL) {
        Visit(root->Value()); //访问当前结点
        //周游头一棵树树根的子树
        RootFirstTraverse(root->LeftMostChild());
        root=root->RightSibling();//周游其他的树
      }
}
```



后根深度优先周游树(森林)

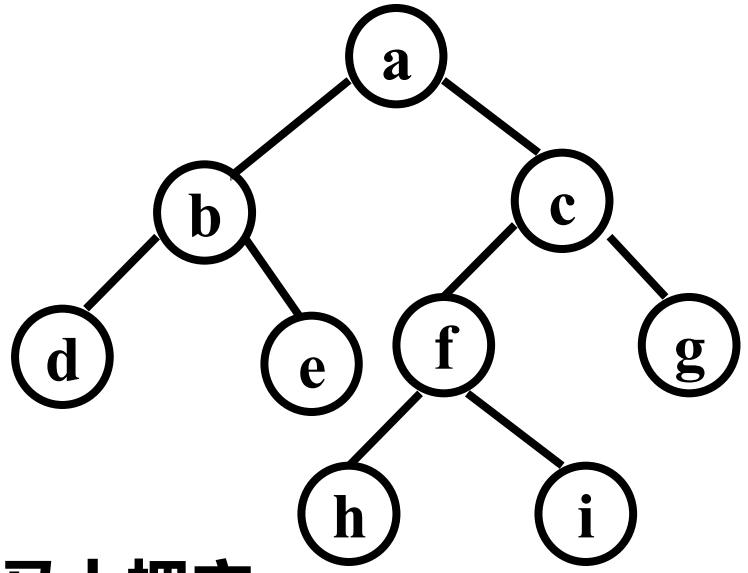
```
template <class T>
void Tree<T>::RootLastTraverse (TreeNode<T>* root){
  while (root !=NULL) {
      //周游头一棵树树根的子树
      RootLastTraverse (root->LeftMostChild());
      Visit (root->Value());//访问当前结点
      root=root->RightSibling();//周游其他的树
  }
}
```



- 递归调用 RootLastTraverse(B) (A的第一个子树)
  - RootLastTraverse(E) (B的第一个子树)
    - E没有左孩子，访问 E
    - E没有右兄弟，返回
  - 访问 B
  - 移动到B的右兄弟C
    - RootLastTraverse(F) (C的第一个子树)
      - F没有左孩子，访问 F
      - F没有右兄弟，返回
    - 访问 C
    - 移动到C的右兄弟D
      - RootLastTraverse(G) (D的第一个子树)
        - G没有左孩子，访问 G
        - G没有右兄弟，返回
      - 访问 D
      - D没有右兄弟，返回
- 访问 A
- A没有右兄弟，结束

广度优先周游树(森林)

- 先访问层数为0的结点；然后从左到右逐个访问层数为1的结点；……；依此类推，直到访问完树中的全部结点
- 实现



  - 可以利用一个队列实现其算法
  - 首先把被周游的树根送入队列
  - 其后，每当从队首取出一棵树，访问其根结点之后，马上把它的子树按从左到右的次序送入队列尾端
  - 重复此过程直到队列为空

广度优先周游树(森林)算法

```
template<class T>
void Tree<T>::WidthTraverse(TreeNode<T> * root)  {
    using std::queue;                                    // 使用STL队列
    queue<TreeNode<T>*> aQueue;
    TreeNode<T> * pointer = root;
    while (pointer != NULL) {
        aQueue.push(pointer);                    // 当前结点进入队列
        pointer = pointer->RightSibling(); //指向结点的右兄弟
    }
```

```
    while (!aQueue.empty()) {
        pointer = aQueue.front();  // 获得队首元素
        aQueue.pop();       // 当前结点出队列
        Visit(pointer->Value());  // 访问当前结点
        pointer = pointer-> LeftMostChild();
        // pointer指向当前结点的最左孩子
        while (pointer != NULL) {//当前结点的子结点进队列
            aQueue.push(pointer);
            pointer = pointer->RightSibling();
        }
    }
}
```

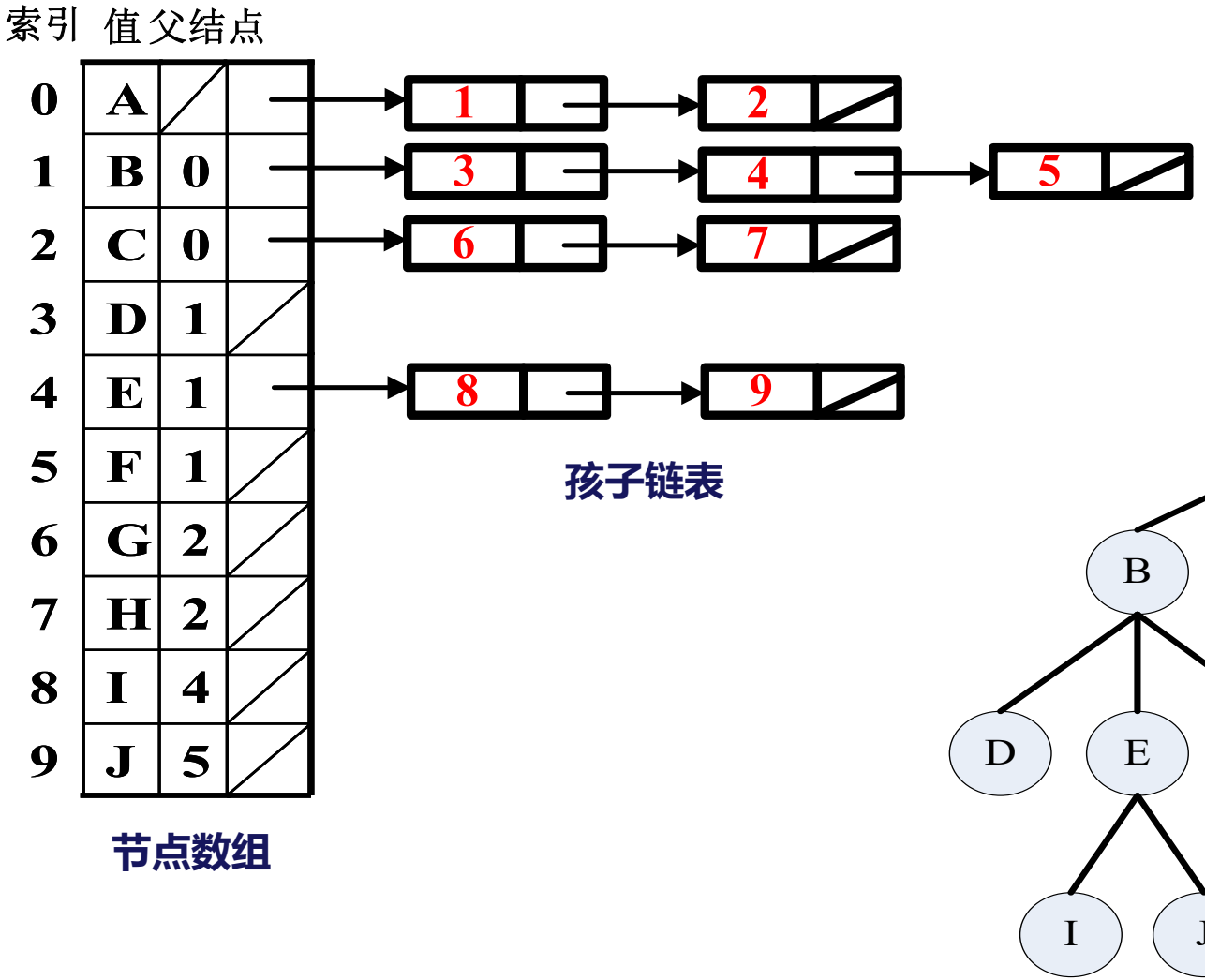
课程内容

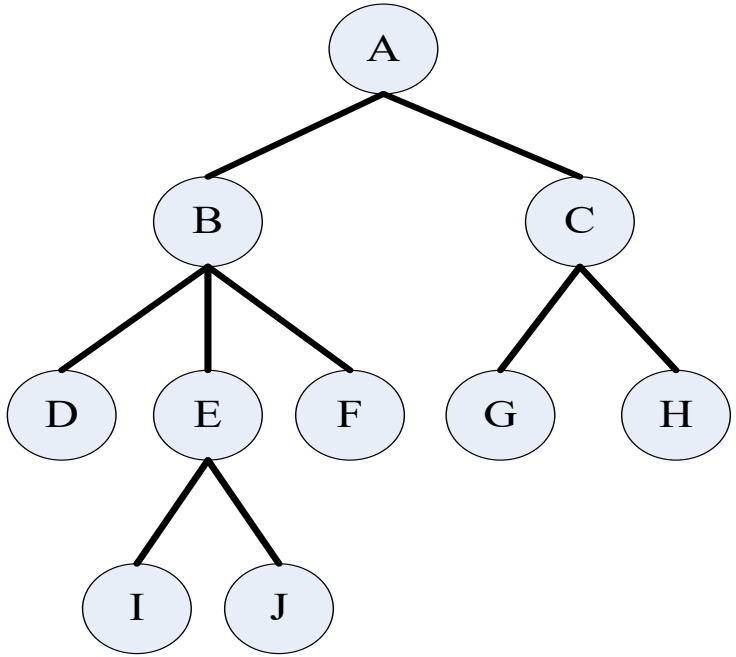
- 6.1 基本概念
- 6.2 链式存储
- 6.3 顺序存储
- 6.4 K叉树

6.2 树的链式存储

- 子结点表表示法
- 动态结点表示法
- 静态“左子结点/右兄弟结点”表示法
- 动态“左子结点/右兄弟结点”表示法
- 父指针表示法及在并查集中的应用

1、子节点表表示法

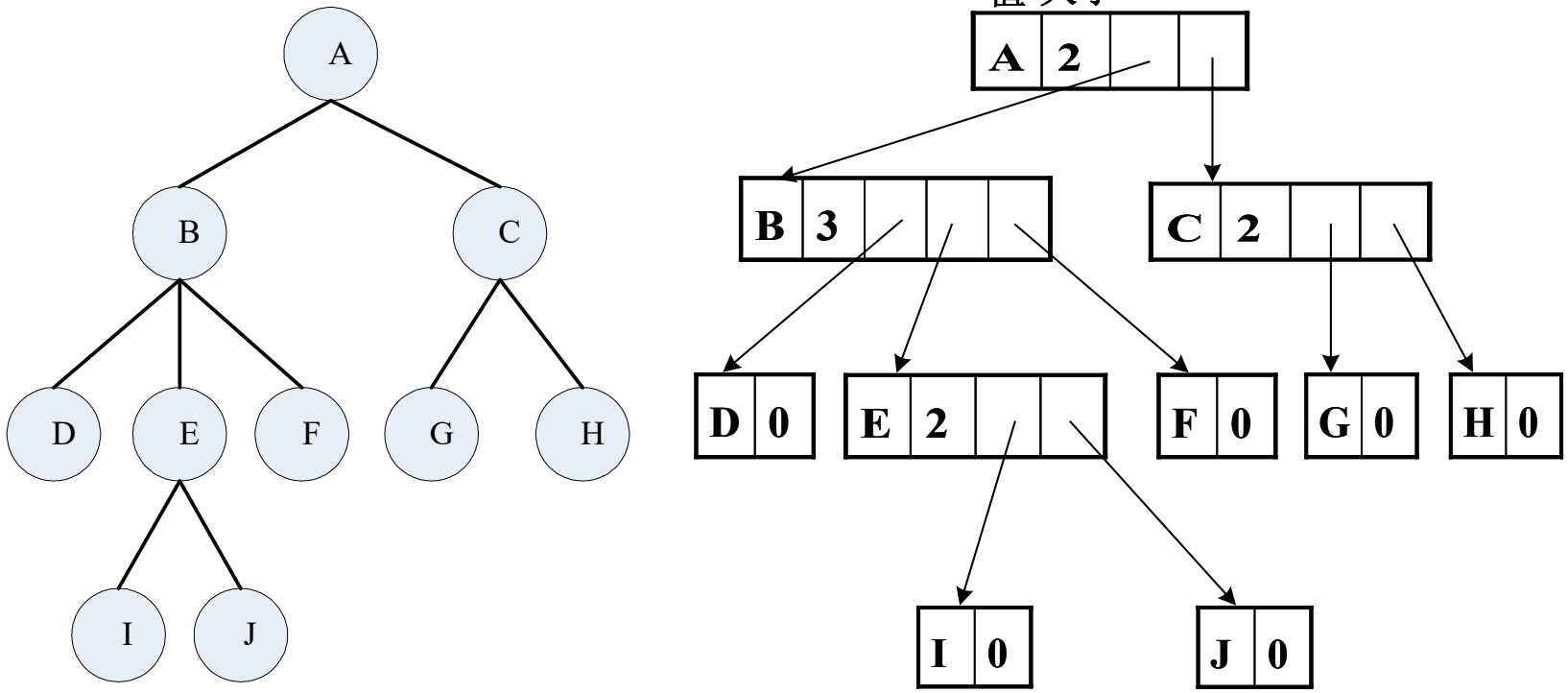

索引 值 父结点
0 A
1 B 0
2 C 0
3 D 1
4 E 1
5 F 1
6 G 2
7 H 2
8 I 4
9 J 5
1
2
3
4
5
6
7
8
9
孩子链表
节点数组


A
B
C
D
E
F
G
H
I
J

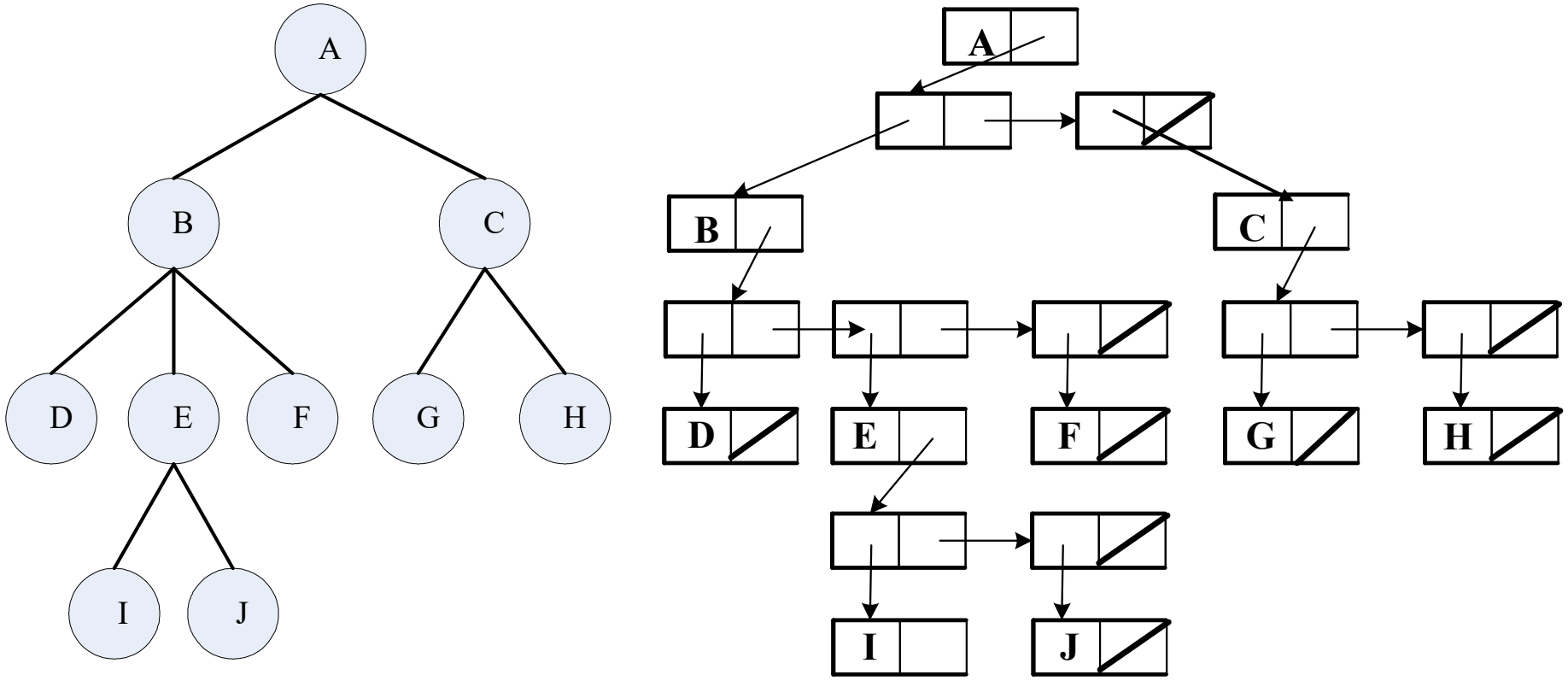
分析

- 优点
  - 查孩子个数和结点的值容易
- 缺点
  - 找兄弟结点困难
- 树的归并容易
  - 只需一棵树的根添到另一棵树的孩子结点表中即可

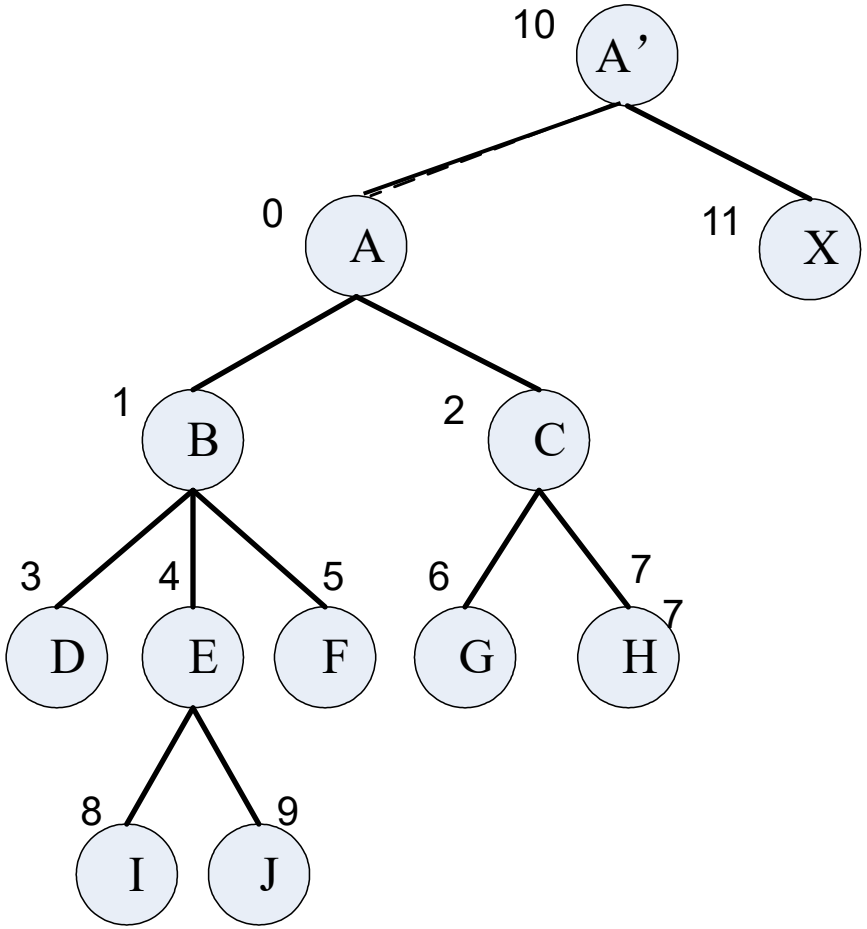
2、动态结点表示法：指针数组法



指针链表法



3、静态“左孩子/右兄弟”表示法



	左子结点	值	父结点	右兄弟结点
1	1	A	10	11
3	3	B	0	2
6	6	C	0	
		D	1	4
8	8	E	1	5
		F	1	
		G	2	7
		H	2	
		I	4	9
		J	4	
0	0	A’		
		X	10	

分析

树的合并

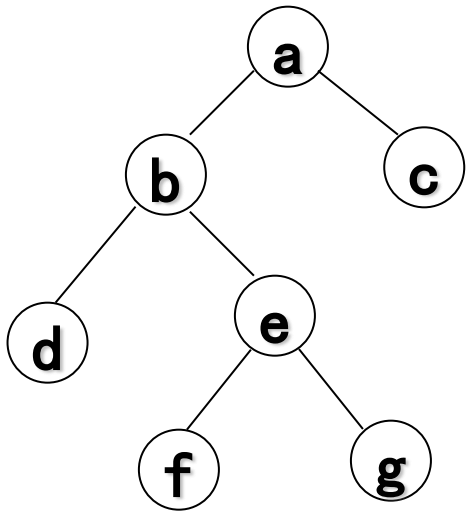
- 如果两棵树存储在同一个数组中，那么把其中一个添加为另一棵树的子树只需简单设置指针值即可

优点

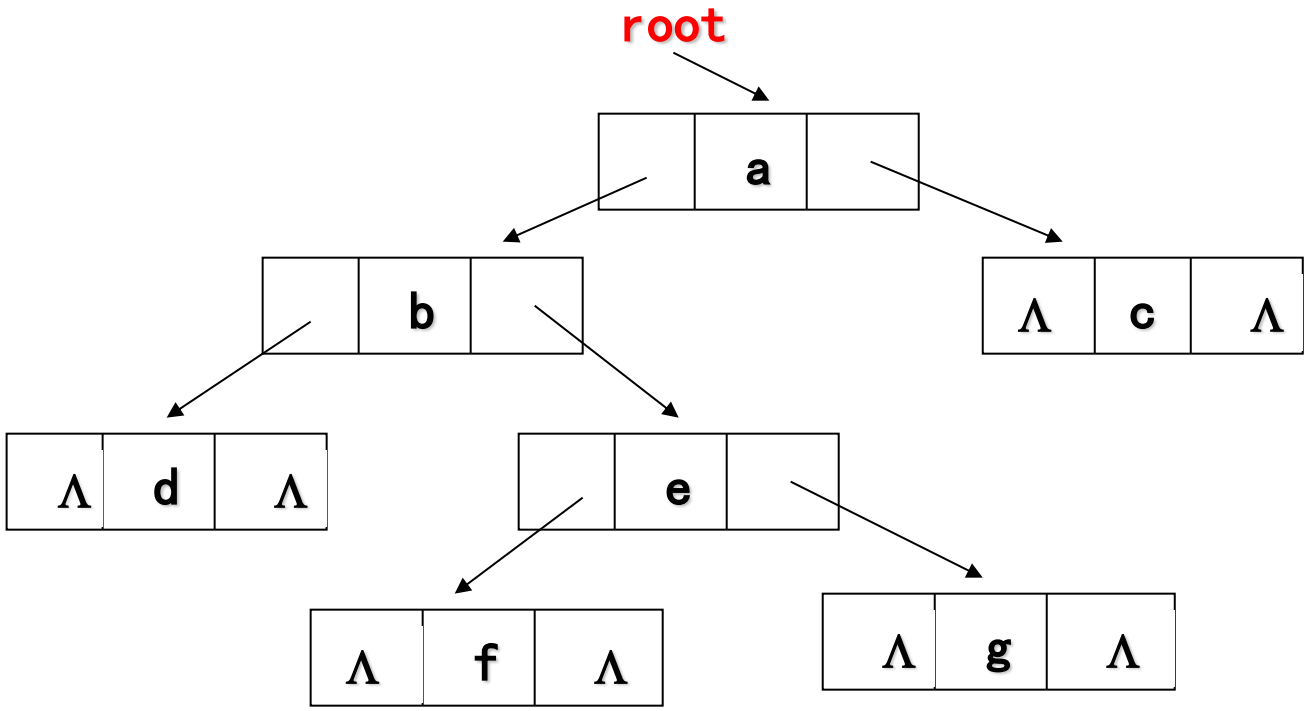
- 比子结点表表示法空间效率更高
- 结点数组中的每个结点仅需要固定大小的存储空间

4、动态“左孩子/右兄弟”二叉链表表示法

- 本质上，使用二叉树来替换树



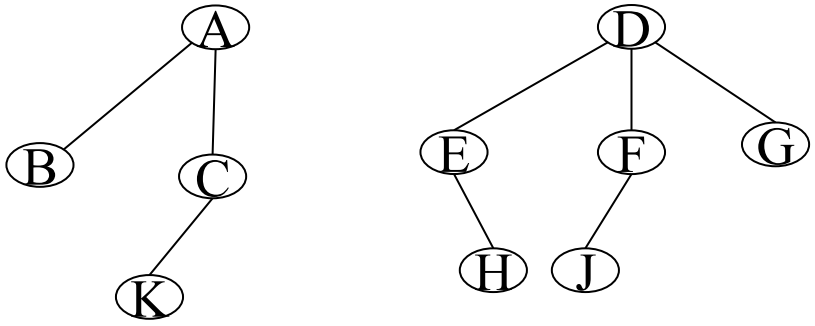
(a) 二叉树

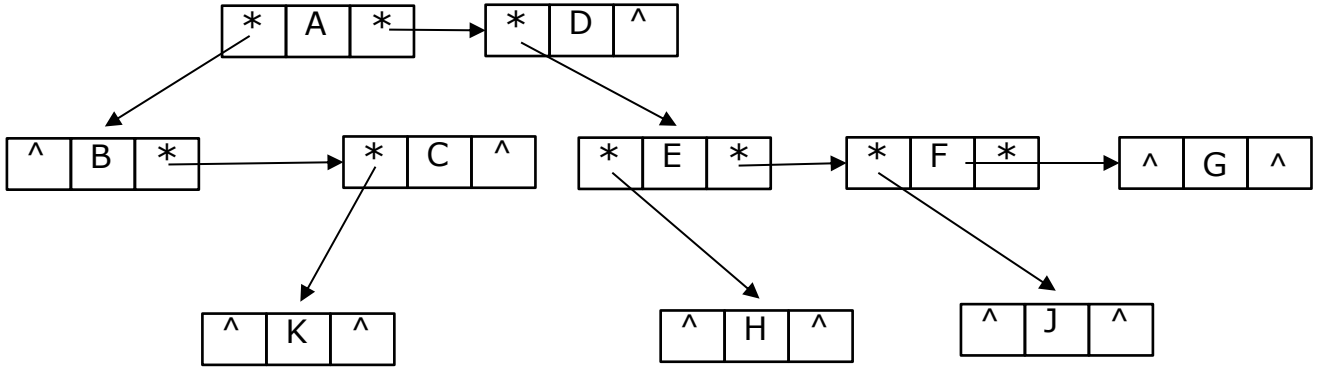


(b) 二叉树的链式存储

基本思想

- 左孩子在树中是结点的最左子结点，右子结点是结点原来的右侧兄弟结点
- 根的右链就是森林中每棵树的根结点



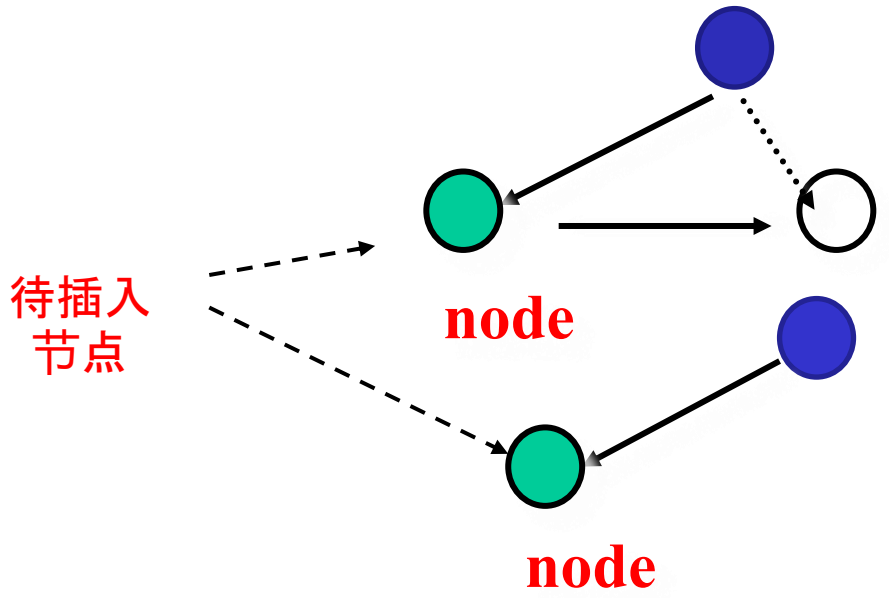


树结点抽象数据类型的实现

```
private: //补充与具体实现相关的私有成员变量申明
    T m_Value;                          //树结点的值
    TreeNode<T>*    pChild;             //左孩子
    TreeNode<T>*    pSibling;           //右兄弟

//公有成员函数的具体实现
template<class T> bool TreeNode<T>::isLeaf()
{//如果结点是叶，返回true
    if(pChild==NULL)
        return true;
    return false;
}
```

```
template<class T>
void TreeNode<T>::InsertFirst(TreeNode<T>* node)
{//以第一个子结点的身份插入结点
    if(pChild) {
        node->pSibling=pChild;
        pChild=node;
    }
    else
        pChild=node;
}
```



树抽象数据类型的实现

```
//与具体实现相关的私有成员变量与成员函数的申明
private:
    TreeNode<T>* root;                    //树根结点

    //返回current的父节点
    TreeNode<T>* Parent(TreeNode<T>*current);
     //删除以root为根的子树的所有结点  (root为原始森林中第一颗树的根)
    void Tree<T>::DestroyNodes(TreeNode<T> *root)
    //删除以subroot为根的子树的所有结点  (subroot为原始森林中的任何一个节点)
    void Tree<T>::DeleteSubTree(TreeNode<T> *subroot)
```

```
template  <class T>   //利用广度优先遍历的方法来进行判定!
TreeNode<T>* Tree<T>::Parent(TreeNode<T> *current)  {
    using std::queue;  // 使用STL队列
    queue<TreeNode<T>*> aQueue;
    TreeNode<T> *pointer = root;
    TreeNode<T> *upperlevelpointer = NULL;      // 用于记录parent结点
    if (current != NULL && pointer != current) {
        while (pointer) {                              // 森林中所有根结点进队列
            if (current == pointer)
                return NULL;                  // 根的父结点指针为空，返回
            aQueue.push(pointer);
            pointer=pointer-> RightSibling();
        }
```

```
        while (!aQueue.empty()) {
                pointer = aQueue.pop();          // 出队列
                upperlevelpointer = pointer;     // 指向上一层的结点
                pointer = pointer-> LeftMostChild(); // 指向最左孩子
                while (pointer) {                    // 当前结点的子结点进队列
                        if (current == pointer)
                                return upperlevelpointer; // 返回父结点指针
                        else {
                                aQueue.push(pointer);
                                pointer = pointer->RightSibling();
                        }
                }//end while
          }//end while
    }//end if
    return NULL;
}
```

```
template <class T>
void Tree<T>::DestroyNodes(TreeNode<T>* root)
{//删除以root为代表的森林的所有结点
    if  (root)
    {
        //递归删除第一子树
        DestroyNodes(root->LeftMostChild());
        //递归删除其他子树
        DestroyNodes(root->RightSibling());
        delete root;            //删除根结点
    }
}//后续遍历方式进行树的删除!
```

```
template <class T>
void Tree<T>::DeleteSubTree(TreeNode<T>* subroot){
//删除以subroot为根的子树的所有结点
    TreeNode<T>* pointer=PrevSibling(subroot); //如何实现?
    if(pointer==NULL) {
            pointer=Parent(subroot);
            if(pointer) {        //父结点pointer存在的情况下
                pointer->pChild=subroot->RightSibling();
                subroot->pSibling=NULL;
            }
            else  {              //父结点pointer不存在的情况下
                root=subroot->RightSibling();
                subroot->pSibling=NULL;
            }
    }//end if
```

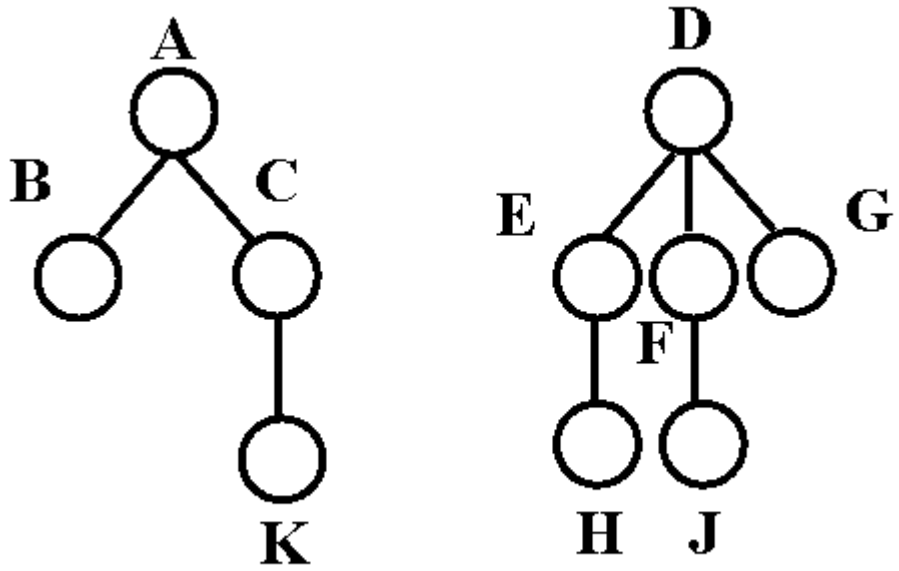
subroot为最左子结点的情况

```
        else{//subroot不是最左子结点
                //链接到删除结点后的结点
                pointer->pSibling=subroot->rightSibling();
                subroot->pSibling=NULL;
        }
        DestroyNodes(subroot);    //销毁以subroot为根的子树
}
```

6.2.5 父指针表示法及在并查集中的应用

- 在某些应用中，只需要知道父结点情况，因此每个结点只需要保存一个指向其父结点的指针域，这种实现被称为**父指针**(parent pointer)**表示法。**
- 用数组存储树所有结点，同时在每个结点中附设一个“指针” 指示其父结点的位置。
- 由于树中每一个结点的父指针是唯一的，所以父指针表示法**可以唯一表示一棵树。**

父指针数组表示法



父节点索引	/	0	0	2	/	4	4	4	5	6
标记	A	B	C	K	D	E	F	G	H	J
结点索引	0	1	2	3	4	5	6	7	8	9

- **查询结点的根**
  - 从一个结点出发找出一条向上延伸到达根的祖先路径
    - O(k)，k为树高
- **判断两个结点是否在同一棵树**
  - 两个结点根结点相同，它们一定在同一棵树中
  - 如果其根结点不同，那么两个结点就不在同一棵树中

分析

- 优点
  - 寻找父结点只需O(K)时间
  - 求树根结点非常方便
- 缺点
  - 寻兄弟节点麻烦，需要查询整个树结构
  - 没有标识节点的左右次序，适合无序树的情况

并查集

- 并查集是一种特殊集合，由不相交子集构成
- 基本操作
  - Different：判断两个结点是否在同一个集合中
  - Union：归并两个集合
- 并查集可用于求解等价类问题

等价关系

- 一个具有 n 个元素的集合 S，另有一个定义在集合 S 上的 r 个关系的关系集合 R。x，y，z表示集合中的元素
- 若关系 R 是一个 **等价关系**，当且仅当如下条件为真时成立：
  - (a) 对于所有的 x，有 (x，x)∈R（即关系是**自反**的）
  - (b) 当且仅当 (x，y)∈R 时 (y，x)∈R（即关系是**对称**的）
  - (c) 若 (x，y)∈R 且 (y，z)∈R，则有 (x，z)∈R（即关系是**传递**的）
- 如果(x，y)∈R，则元素x和y是等价的

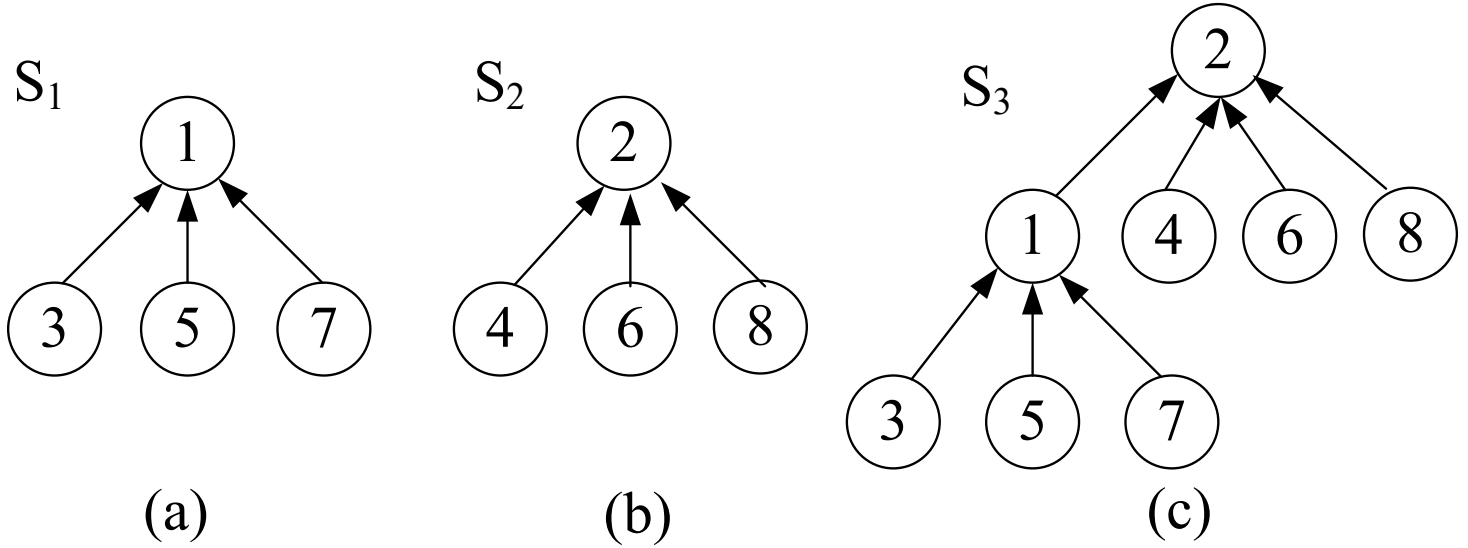
等价类的求解问题

- 等价关系具有自反性、对称性和传递性
- 并查算法可以很容易地解决等价类问题
  - 开始时，每个元素都在独立的只包含一个结点的树中，而它自己就是根结点
  - 通过使用函数Different，判断一个等价对中的两个元素是否在同一棵树中
    - 如果是，由于它们已经在同一个等价类中，不需要作变动
    - 否则两个等价类可以用UNION函数归并

并查算法

- 父指针表示法常常用来维护由一些不相交子集构成的集合
- 用一棵树代表一个集合
- 两种基本操作
  - 判断两个结点是否在同一个集合中，使用Different算法
  - 归并两个集合，使用UNION算法
- 如果两个结点在同一棵树中，则认为它们在同一个集合中。树中的每个结点（除根结点以外）有仅且有一个父结点

等价类



集合的表示方法

树结点的ADT

```
template<class T>
    class ParTreeNode     {                          //树结点定义
private:
    T       value;                                   //结点的值
    ParTreeNode<T>* parent;                          //父结点指针
    int                 nCount;                      //以此结点为根的子树的总结点个数
public:
    ParTreeNode();                                   //构造函数
    virtual ~ParTreeNode(){};                        //析构函数
    T       getValue();                              //返回结点的值
    void  setValue(const T& val);                    //设置结点的值
    ParTreeNode<T>*  getParent();                    //返回父结点指针
    void  setParent(ParTreeNode<T>* par);            //设置父结点指针
    int     getCount();                              //返回结点数目
    void  setCount(const int count);                 //设置结点数目
}
```

树的ADT

```
template<class T>
    class ParTree {//树定义
public:
    ParTreeNode<T>* array;          //存储树结点的数组
    int    Size;                    //数组大小
    ParTree(const int size);        //构造函数
    virtual ~ParTree();             //析构函数
    //查找node结点所属子树的根结点
    ParTreeNode<T> *Find(ParTreeNode<T>* node) const;
    void Union(int i,int j);        //把下标为i，j的结点所属子树合并
    bool Different(int i,int j);    //判定下标为i，j的结点是否在一棵树中
};
```

成员函数

```
template <class T>
ParTree<T>::ParTree(const int size)  //构造函数
{
    Size=size;
    array=new ParTreeNode<T>[size];
}

template <class T>    //析构函数
ParTree<T>::~ParTree()
{
    delete []array;
}
```

```
template <class T>                          //查找node结点所属子树的根结点
ParTreeNode<T>*ParTree<T>::Find(ParTreeNode<T>* node) const{
    ParTreeNode<T>* pointer=node;
    while ( pointer->getParent()!=NULL)
          pointer=pointer->getParent();
    return pointer;
}
template<class T>                           //判断两个结点是不是属于同一棵子树
bool ParTree<T>::Different(int i,int j){
    ParTreeNode<T>* pointeri=Find(&array[i]);          //找到结点i的根
    ParTreeNode<T>* pointerj=Find(&array[j]);          //找到结点j的根
    return pointeri!=pointerj;
}
```

```
template<class T> void ParTree<T>::Union(int i,int j){
    //将结点i和结点j所属的树合并成为一棵树
    ParTreeNode<T>* pointeri=Find(&array[i]);   //找到结点i的根
    ParTreeNode<T>* pointerj=Find(&array[j]);   //找到结点j的根
    if(pointeri!=pointerj){
        if(pointeri->getCount()>=pointerj->getCount()){
            pointerj->setParent(pointeri);
            pointeri->setCount(pointeri->getCount()+pointerj->getCount());
        }
        else{
            pointeri->setParent(pointerj);
            pointerj->setCount(pointeri->getCount()+pointerj->getCount());
        }
    }//end if
}
```

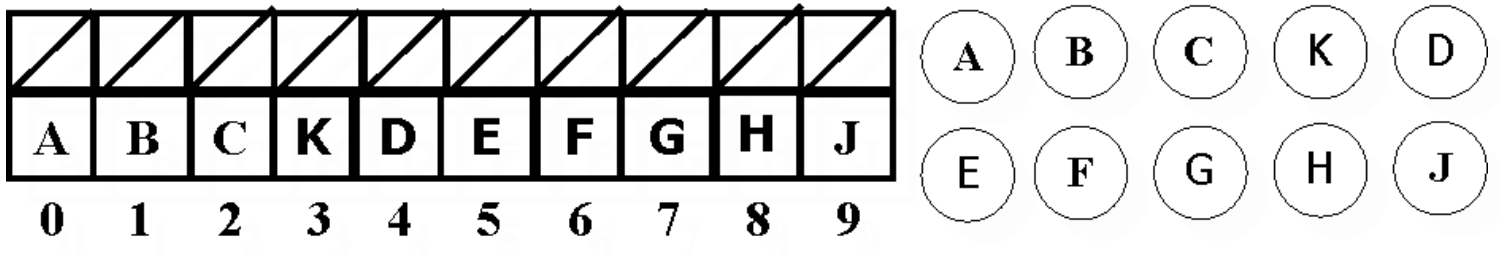
重量权衡合并规则

- “重量权衡合并规则” (weighted union rule)
  - 将结点较少树的根结点指向结点较多树的根结点
  - 这可以把树的整体深度限制在O(log n)
  - 当处理完n个元素后，任何结点的深度最多只会增加logn次
    - 每次归并，最大高度最多增加1
    - 而结点个数成倍增加

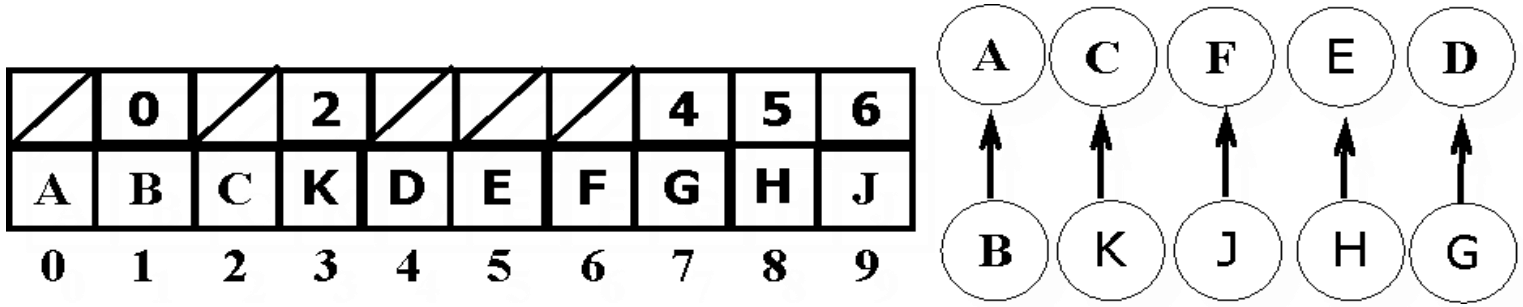
示例

- 10个结点A、B、C、D、E、F、G、H、J、K和它们的等价关系

  (A,B)、(C,K)、(F,J)、(E,H)、(D,G)、(K,A)、(E,G)、(H,J)

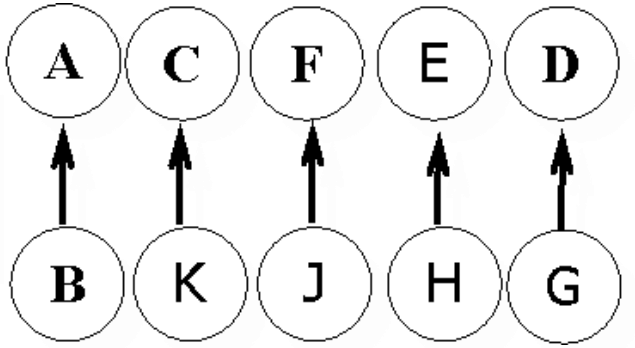


- 对等价对(A,B)、(C,K)、(F,J)、(E,H)、(D,G)的处理结果

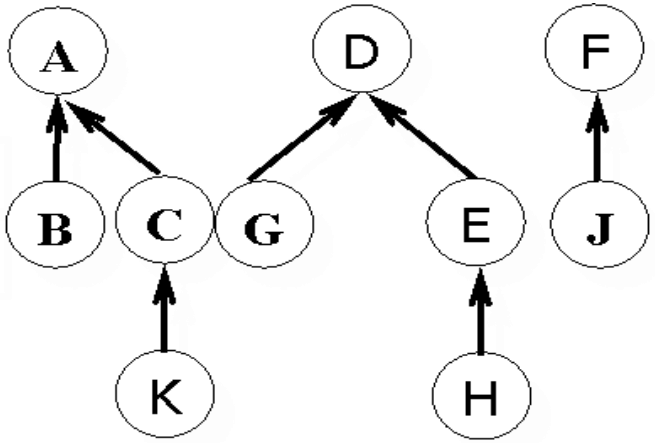


对两个等价对（K，A）和（E，G）的处理结果

	0		2				4	5	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9

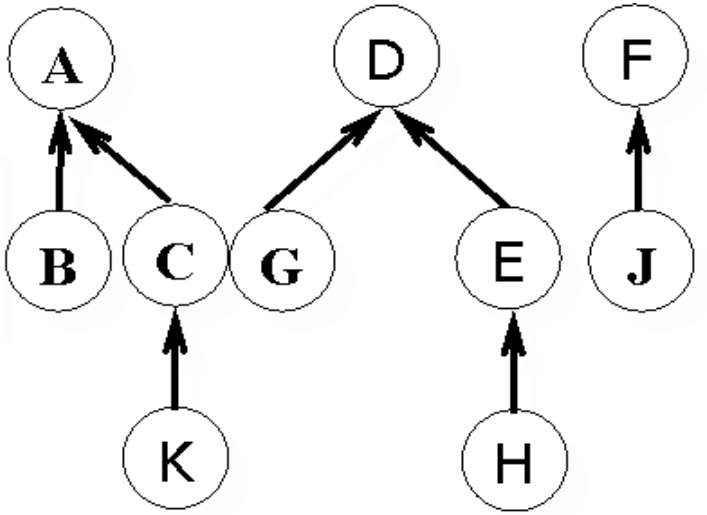


	0	0	2		4		4	5	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9

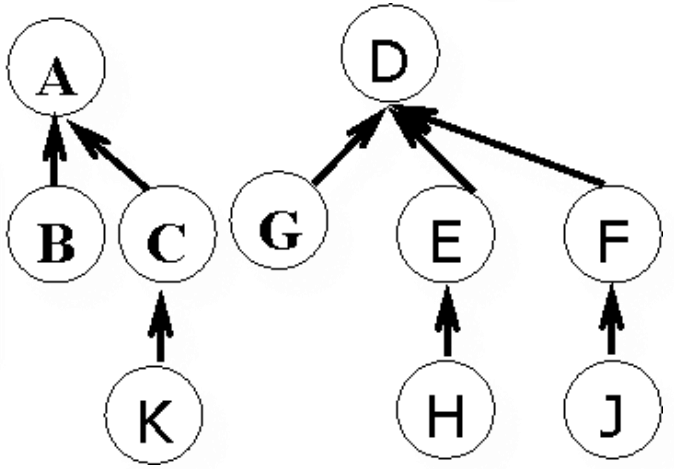


使用标准重量权衡合并规则处理等价对(H, J)的结果

	0	0	2		4		4	5	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9



	0	0	2		4	4	4	5	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9



路径压缩算法

```
template <class T>   //带路径压缩的find算法
ParTreeNode<T> *ParTree<T>::FindPC(ParTreeNode<T>* node)
    const  {
    if(node->getParent()==NULL)
            return node;
    node->setParent(FindPC(node->getParent()));
    return node->getParent();
}
```

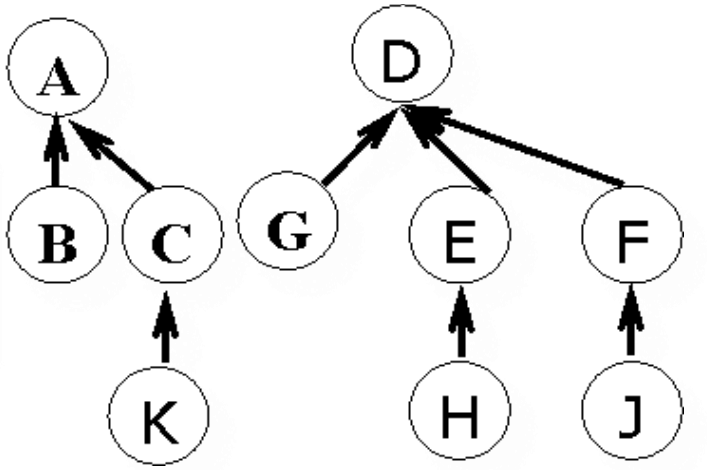
路径压缩算法

- 查找X的根
  - 设X最终到达根R
  - 顺着由X到R的路径把每个结点的父指针域均设置为直接指向R
- 产生极浅树

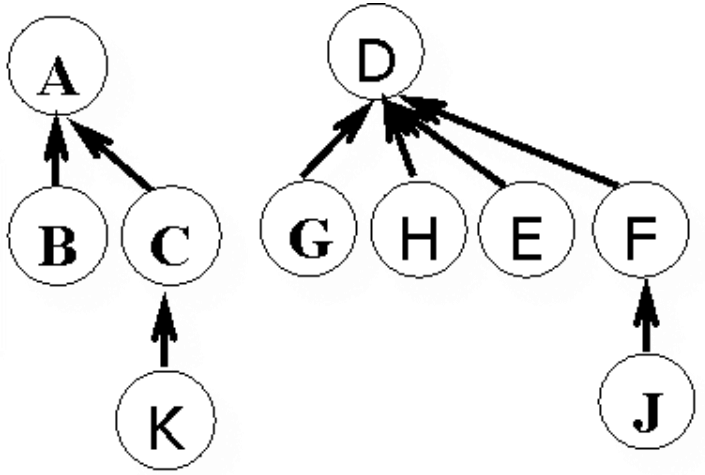
路径压缩示例

- 使用路径压缩规则处理等价对（H,E）的结果

	0	0	2		4	4	4	5	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9



	0	0	2		4	4	4	4	6
A	B	C	K	D	E	F	G	H	J
0	1	2	3	4	5	6	7	8	9



复杂度

假设同时使用了“重量权衡合并规则”和“路径压缩”，则有如下的复杂度：

	AVG	Worst Case
Space	O(n)	O（n）
FIND	O(α(n))	O(α(n))
UNION	O(α(n))	O(α(n))

O(α(n)) 是一个增长非常缓慢的Ackermann函数;
可以认为α(n)是一个小于5的常数。

课程内容

- 6.1 基本概念
- 6.2 链式存储
- **6.3 顺序存储**
- **6.4 K叉树**

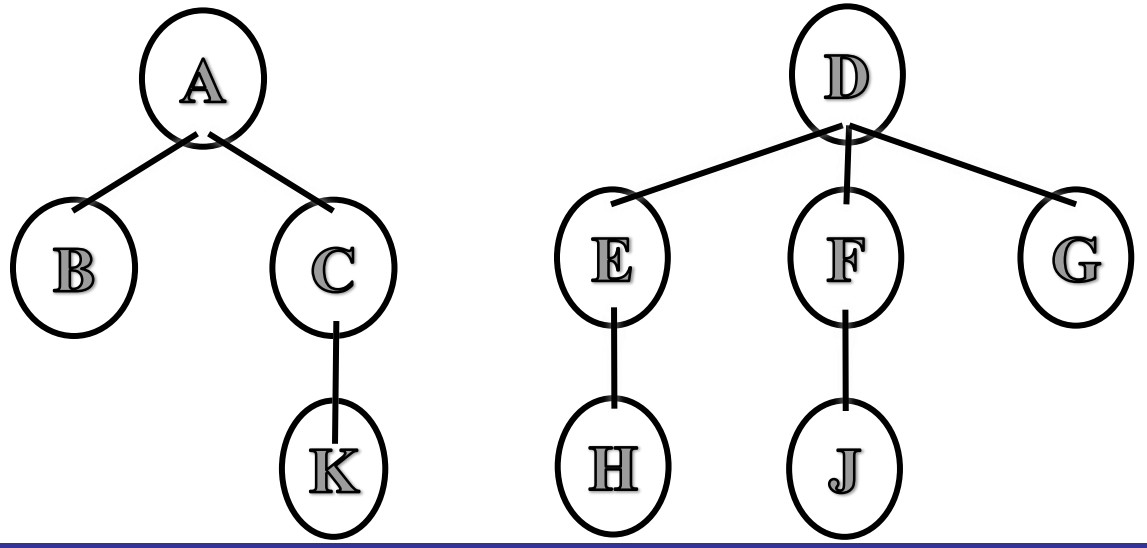
顺序存储方法

- 按照树遍历的次序进行节点存储
  - 带右链的先根次序表示法
  - 带双标记位的先根次序表示法
  - 带度数的后根次序表示法
  - 带双标记的层次次序表示
- 如何还原出树的结构是关键

6.3.1 带右链的先根次序表示法

- 先根遍历树，结果有如下特点
  - 任何结点的子树的所有结点都直接跟在该结点之后
  - 每棵子树的所有结点都聚集在一起，中间不会插入别的结点
  - 任何一个分支结点后面跟的都是它的第一个子结点（如果存在的话）
- 带右链的先根次序表示中，结点按先根次序顺序存储在一片连续的存储单元中
- 但还需如下信息支持
  - 孩子关系如何确定?
  - 兄弟关系如何确定?

A B C K D E H F J G

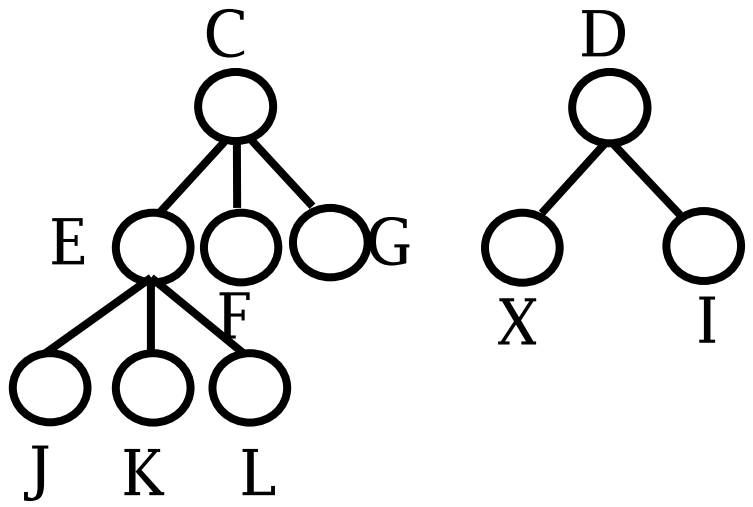


- 结点除包含本身数据外，还附加两个表示结构的信息字段
  - rlink：是右指针，指向下一个兄弟
  - ltag是一个1位的左标记
    - 0：有子结点
    - 1：没有子结点

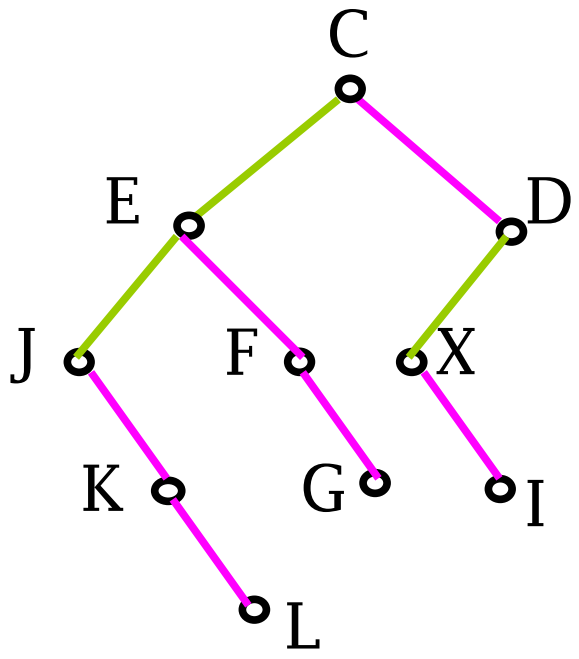


- 与二叉链表相比，用ltag代替llink，占用存储单元少，但并不丢失信息
- 可以从结点的次序和ltag的值完全可以推知llink
  - ltag为0：结点有左子结点，其llink指向存储区中该结点顺序的下一个结点
  - ltag为1：结点没有左子结点，它的llink为空

图示

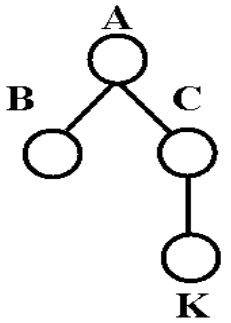


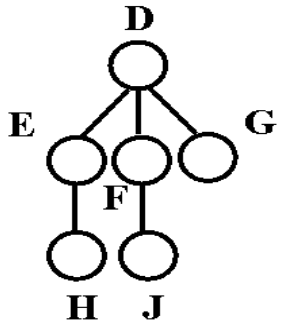
下标	0	1	2	3	4	5	6	7	8	9
rlink	7	5	3	4	-1	6	-1	-1	9	-1
info	C	E	J	K	L	F	G	D	X	I
ltag	0	0	1	1	1	1	1	0	1	1



6.3.2 带双标记位的先根次序表示法

- 事实上，带右链的先根次序表示法中rlink也不是必需的
- 以1位的rtag就足以表示出整个森林的结构信息
  - 当结点无兄弟时，rtag为1，否则为0
- 由结点的排列次序和ltag、rtag的值就可推知rlink的值
  - 当一个结点x的rtag为1时，它的rlink显然应为空
  - 当一个结点x的rtag为0时，它的rlink应指向结点序列中排在以结点x为根的子树中最后结点的后面的那个结点y

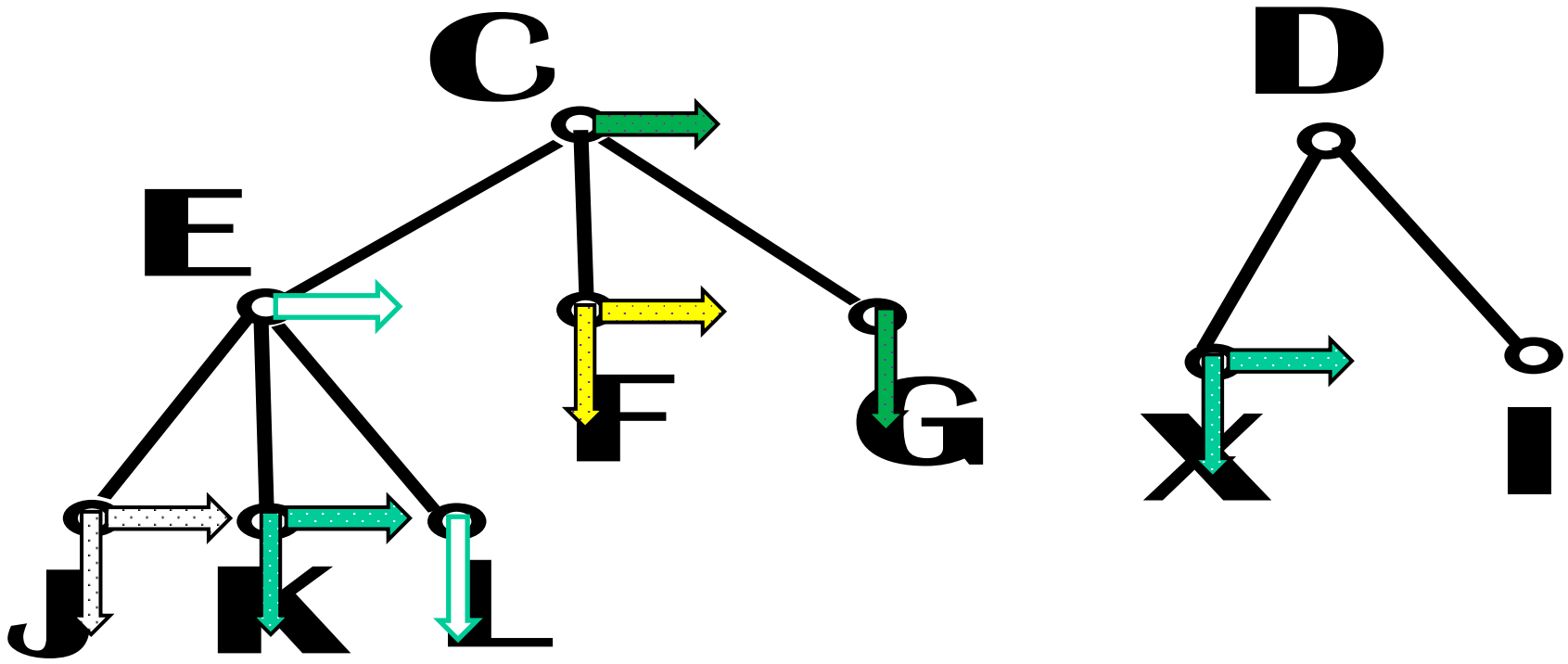




rtag	0	0	1	1	1	0	1	0	1	1
info	A	B	C	K	D	E	H	F	J	G
ltag	0	1	0	1	0	0	1	0	1	1

- 确定结点y成为解决问题的关键！

有兄弟节点与无孩子节点一一对应，满足栈特性



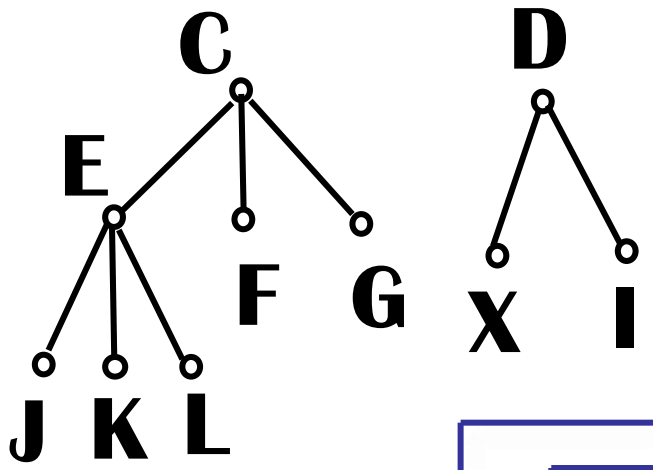
rtag	0	0	0	0	1	0	1	1	0	1
info	C	E	J	K	L	F	G	D	X	I
ltag	0	0	1	1	1	1	1	0	1	1

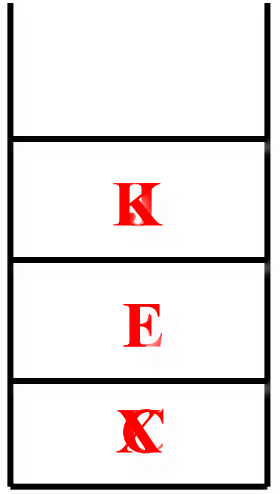
结点x的兄弟结点y的确定方法

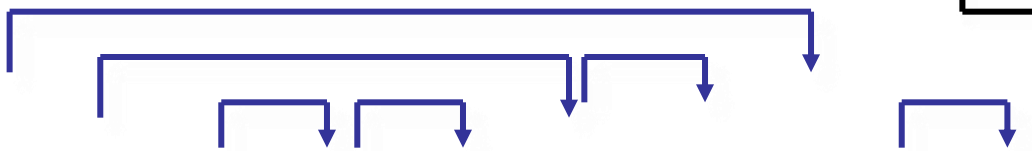
- 由排列次序和ltag，rtag的值推知rlink的值
  - 先根次序中子树结点嵌套出现，在顺序搜索中要嵌套处理x的所有子树，因此确定y要用到栈结构
  - 有兄弟的结点(rtag=0)，都唯一对应一个无孩子的结点(ltag=1)，成对出现，满足栈特性，即
    - 扫描到一个rtag为0的结点就将它进栈
    - 扫描到一个ltag为1的结点就从栈顶弹出一个结点，并为弹出的结点设置rlink为：下一个要读出的节点

“有兄弟就入栈，无孩子就出栈”

- 处理过程中需要用一个栈来记录待配置rlink的结点







rtag	0	0	0	0	1	0	1	1	0	1
info	C	E	J	K	L	F	G	D	X	I
ltag	0	0	1	1	1	1	1	0	1	1

树结点类定义

```
template<class T>
class DualTagTreeNode
{//双标记位先根次序树结点类
    public:
        T       info;                   //树结点信息
        int     ltag;                   //左标记
        int     rtag;                   //右标记
        DualTagTreeNode();              //构造函数
        virtual ~DualTagTreeNode();     //析构函数
};
template<class T> DualTagTreeNode<T>::DualTagTreeNode(){
        ltag=1;  rtag=1;
}
template<class T>DualTagTreeNode<T>::~DualTagTreeNode(){}
```

构造左子结点右兄弟树算法

```
template <class T>
Tree<T>::Tree(DualTagTreeNode<T>* nodeArray, int  count){
    using std::stack;                                          // 使用STL中的stack
    stack<TreeNode<T>* >        aStack;
    TreeNode<T>* pointer=new TreeNode<T>;          //建立根结点
    root=pointer;
    for(int i=0;i<count-1;i++) {                               //处理一个结点
        pointer->setValue(nodeArray[i].info);
        if(nodeArray[i].rtag = =0)   //有兄弟，则压栈～
            aStack.push(pointer);
        else                                    //无兄弟，兄弟域设为空～
            pointer->setSibling(NULL);
```

处理右兄弟标志位

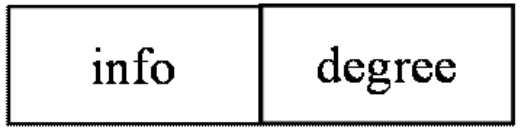
```
        TreeNode<T>* temppointer=new TreeNode<T>;
        if(nodeArray[i].ltag==0)                    //有孩子，则设为孩子~
            pointer->setChild(temppointer);
        else{                                       //无孩子则出栈~
            pointer->setChild(NULL);                //左子结点设为空
            pointer=aStack.pop();
            pointer->setSibling(temppointer);
        }
        pointer=temppointer;
    } //end for
    pointer->setValue(nodeArray[count-1].info);
    pointer->setChild(NULL);
    pointer->setSibling(NULL);
}
```

处理左孩子标志位

处理最后一个结点

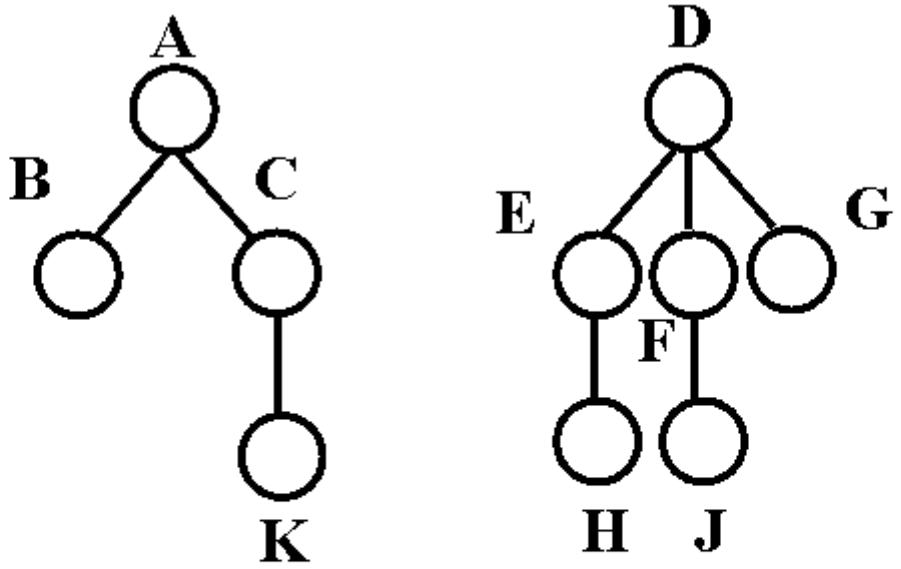
6.3.3 带度数的后根次序表示法

- 结点按后根次序顺序存储在一片连续的存储单元中，结点的形式为
  - 其中，info是结点的数据，degree是结点的度数



- 这种表示法不包括指针，但它仍能反映树结构
- 若某结点的degree值为m，则该结点有m个子结点
  - 最右子结点就是后根次序序列中该结点的前驱
  - 最右第二个子结点是以最右子结点为根的子树在后根次序序列中的前驱
  - …。

树(森林)的图示

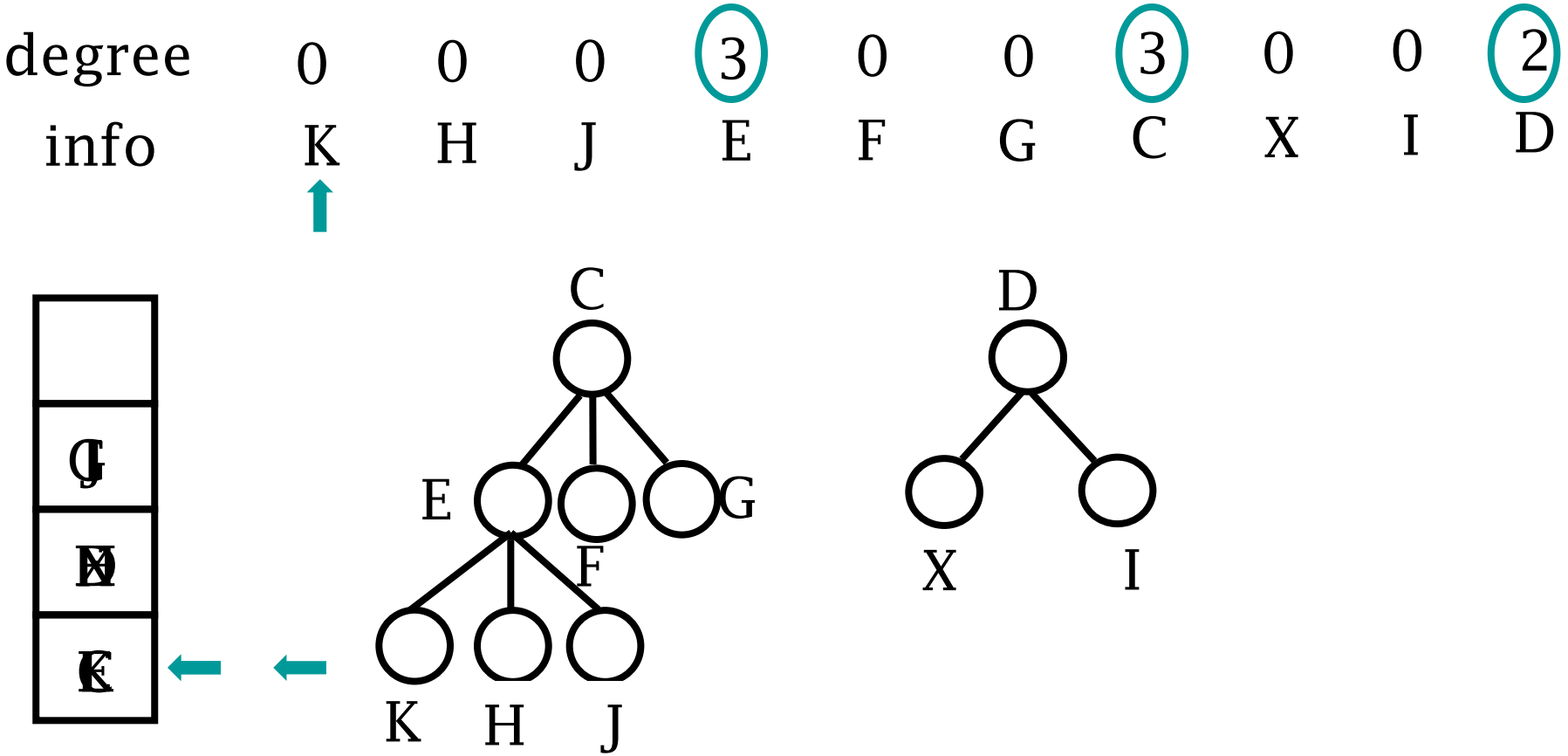


degree	0	0	1	2	0	1	0	1	0	3
info	B	K	C	A	H	E	J	F	G	D

思路

- 将带度数的后根次序表示转化成森林时
  - 只需要从左至右进行扫描，度数为零的结点是叶子结点（也可看作一棵子树）
  - 当遇到度数非零（设为k）的结点时，则排在该结点之前且离它最近的k个子树的根就是该结点的k个子结点
- 利用栈实现
  - 遇到零度顶点就入栈；
  - 遇到非零k度顶点就从栈中弹出k个节点作为其子节点，然后将该非零顶点入栈
  - 持续扫描， 直至序列扫描完毕

示例

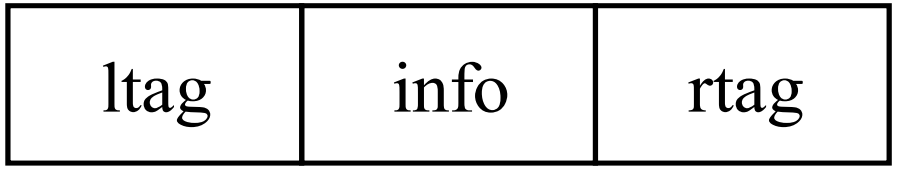


思考题

- 带度数的先根次序?
- 带度数的层次次序?

6.3.4 带双标记的层次次序表示

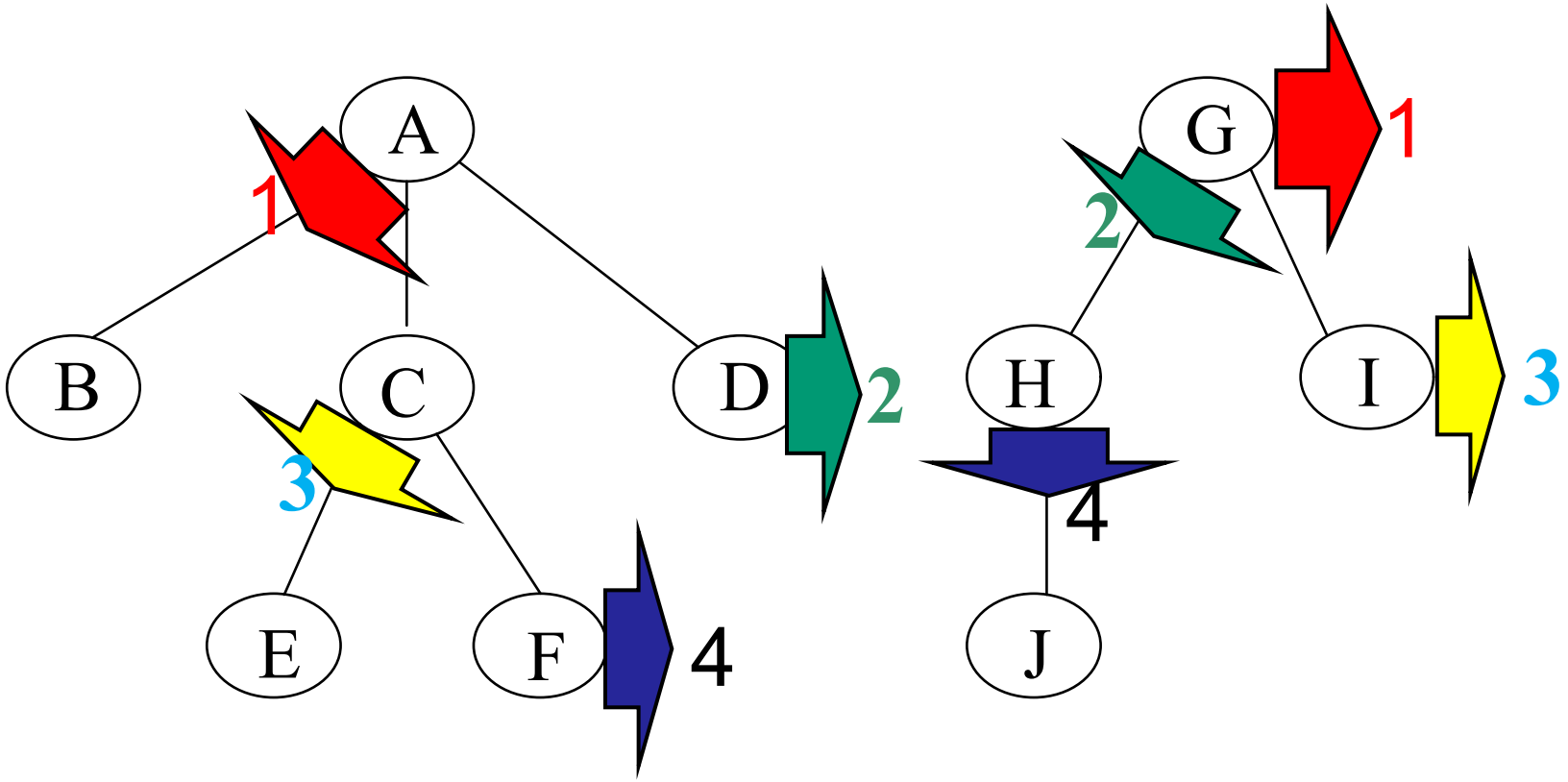
- 类似于带双标记的先根次序表示，引入带双标记的层次次序表示法，其结点的形式为：



  - 当结点没有左孩子时标记位ltag为1，否则为0
  - 当结点没有下一个兄弟时标记位rtag为1，否则为0
    - rtag为0时，下一个节点即为其兄弟节点
    - rtag为1时，无兄弟节点

关键：处理ltag为0的情况

有孩子节点与无兄弟节点一一对应，满足队列特性



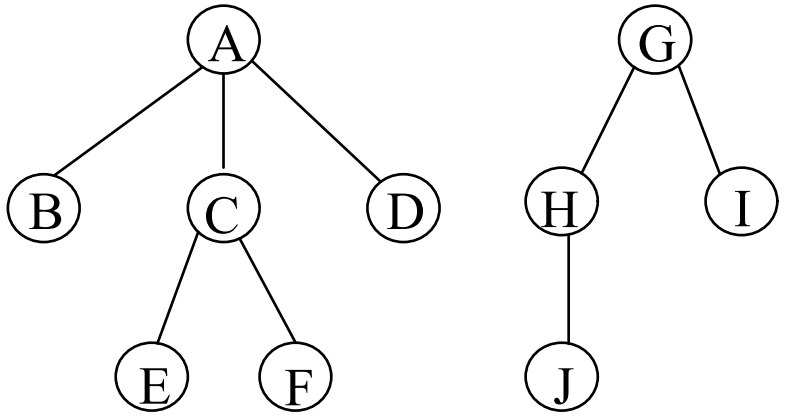
ltag	0	0	1	0	1	0	1	1	1	1
info	A	G	B	C	D	H	I	E	F	J
rtag	0	1	0	0	1	0	1	0	1	1

有孩子则入队列，无兄弟出队列！

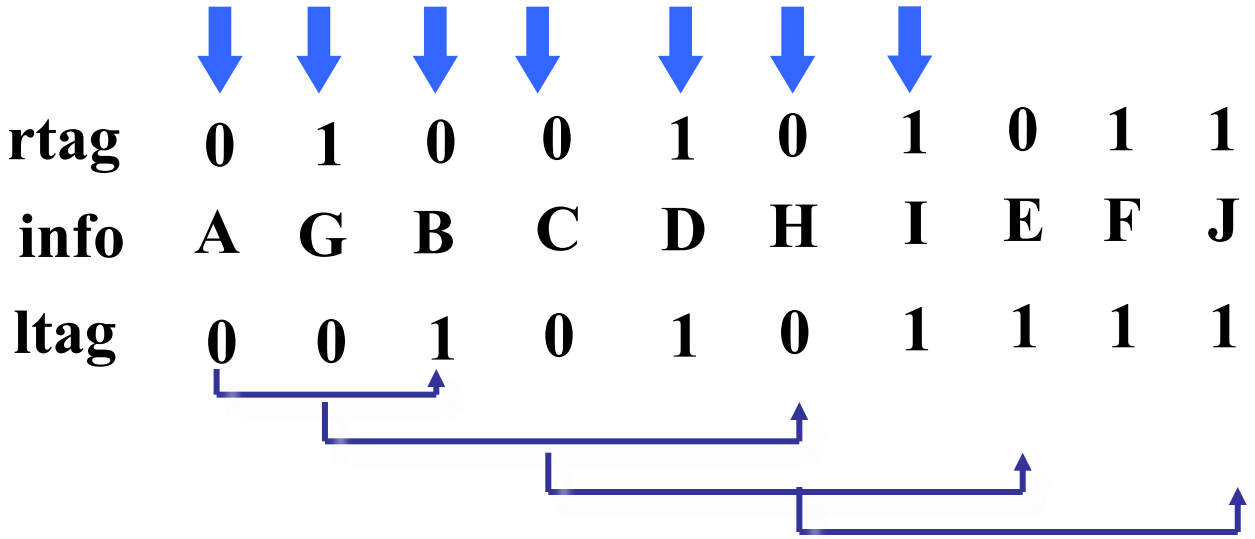
- 顺序扫描带双标记的层次次序序列
  - 如果结点的ltag值为1，则置其llink为空；当结点的ltag为0时，该结点入队列；
  - 如果结点的rtag值为0，那么其后的结点y就是其右兄弟；
  - 否则，如果结点的rtag值为1，则rlink为空，此时出队列x，并将x的llink指向序列中后续结点 y即可。

“有孩子就入队列，无右兄弟就出队列”

- 处理过程中需要用一个队列来记录待配置llink的结点



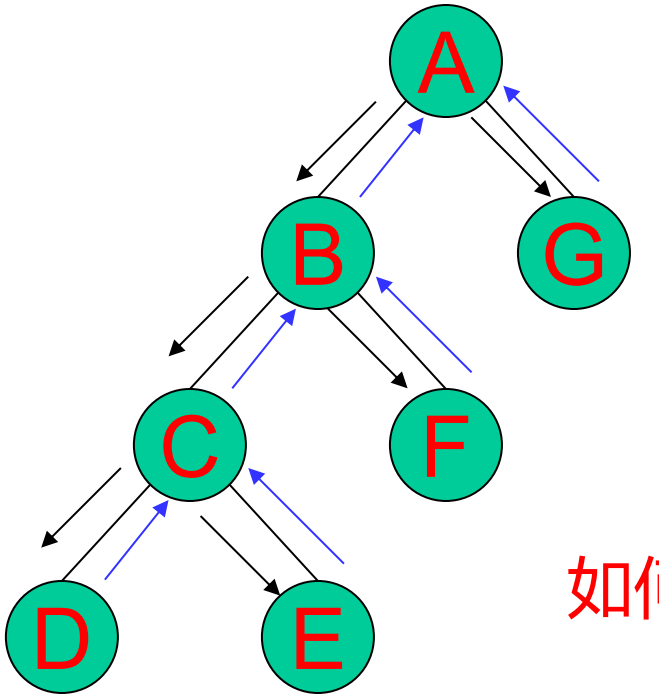
A	G	C	H



与双标记先根遍历代码几乎一样

请自己予以实现！

树的前序字符串表示



A B C D -1 E -1 -1 F -1 -1 G -1

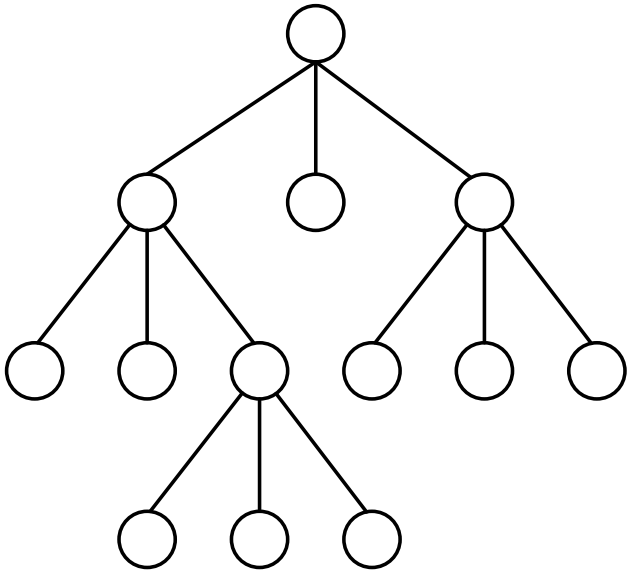
如何得到树的前序字符串表示?

如何从树的前序字符串表示恢复到树结构?

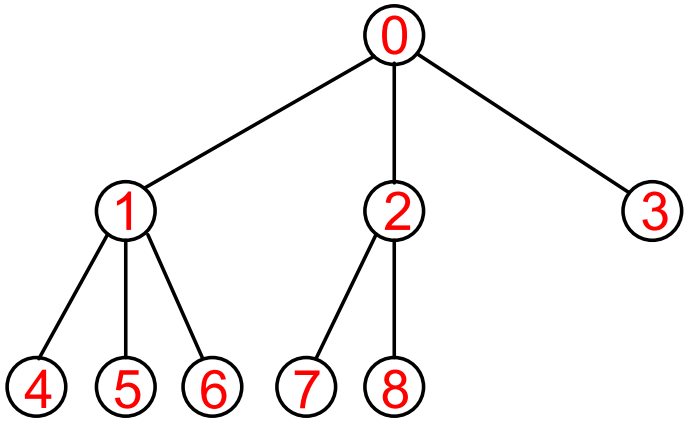
6.4 K叉树

- K叉树(K-ary Tree)的结点有K个有序子结点
- 不同于树，K叉树的结点有K个子结点，子结点数目是固定的，因此相对来说容易实现
- 满K叉树和完全K叉树与满二叉树和完全二叉树是类似的
- 二叉树的许多性质可以推广到K叉树
- 也可以把完全K叉树存储在一个数组中

示例



（a）满3叉树



（b）完全3叉树

知识点总结

- 树和森林的概念
- 树与二叉树的联系、区别与转换
- 树的链式存储
  - “左子结点/右兄弟结点”二叉链表
  - 父指针表示法
- 树的顺序存储
- K叉树

练习

- **若某树有n1 个度数为1 的结点，有n2 个度数为2 的结点，……有nm 个度数为m的结点，试问它有多少个叶结点，给出计算的过程。**

假设叶子结点数为n0,并假设树的结点数为N,

N = n0+n1+n2+...+nm　　又等于所有节点的分支数（或度数）+1。

N = n1+2*n2+3*n3+...+m*nm+1

这样得到n0+n1+n2+...+nm = 1+n1+2*n2+3*n3+...+m*nm

即得：n0 = n2+2*n3+3*n4+...+(m-1)*nm+1

练习

- 如果结点A有3个兄弟(不包括A本身)，而且B是A的父结点，则B的度是（）。

  A.3

  B.4

  C.5

  D.1

练习

- 设高度为h的二叉树上只有度为0和度为2的结点，则此二叉树中所包含的结点数至少为( )。

A. 2h

B. 2h-1

C. 2h+1

D. h+1

练习

- 树最适合用来表示（）

A 有序的数据元素

B 无序的数据元素

C 元素之间具有分支层次关系的数据

D 元素之间无联系的数据

练习

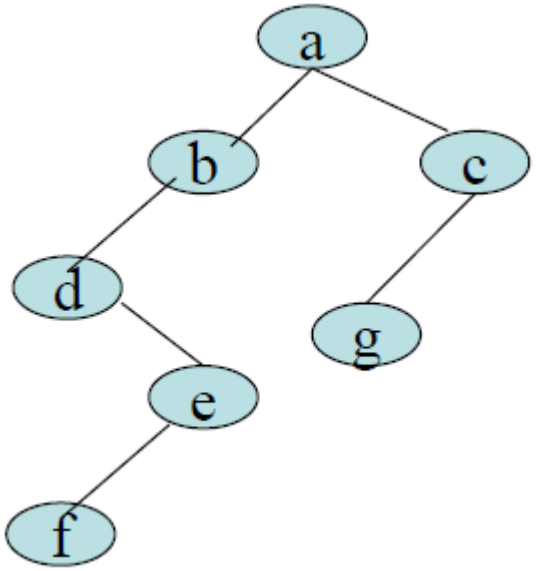
- 如图所示二叉树的中序遍历序列是（）

A abcdgef

B dfebagc

C dbaefcg

D defbagc



练习

- 已知某二叉树的后序遍历序列是dabec，中序遍历序列是debac，它的前序遍历序列是（）。

A acbed

B decab

C deabc

D cedba

练习

- 任何一棵二叉树的叶结点在先序、中序和后序的遍历序列中的相对次序（）

A 不发生改变

B 发生改变

C 不能确定

D 以上都不对

练习

- 某二叉树的先根序列和后根序列正好相反，则该二叉树一定是（）二叉树。

A．空或只有一结点

B．树的高度等于其结点数减1

C．任一结点都只有右子结点

D．任一结点都只有左子结点

练习

- 设n,m为一棵二叉树上的两个结点，在中序遍历时，n在m前的条件是（）

  A. n在m右方

  B. n在m祖先

  C. n在m左方

  D. n在m子孙

练习

- 在一非空二叉树的中序遍历序列中，根结点的右边( )

A.只有右子树上的所有结点

B.只有右子树上的部分结点

C.只有左子树上的所有结点

D.只有左子树上的部分结点

练习

- 某二叉树T有n个结点,设按某种顺序对T中的每个结点进行编号，编号值为1,2,...n.且有如下性质:T中任意结点v,其编号等于左子树上的最小编号减一,而v的右子树的结点中,其最小编号等于v左子树上结点的最大编号加一，这是按( )编号的。

  A.中序（中根）遍历序列

  B.前序（先根）遍历序列

  C.后序（后根）遍历序列

  D.层次顺序

练习

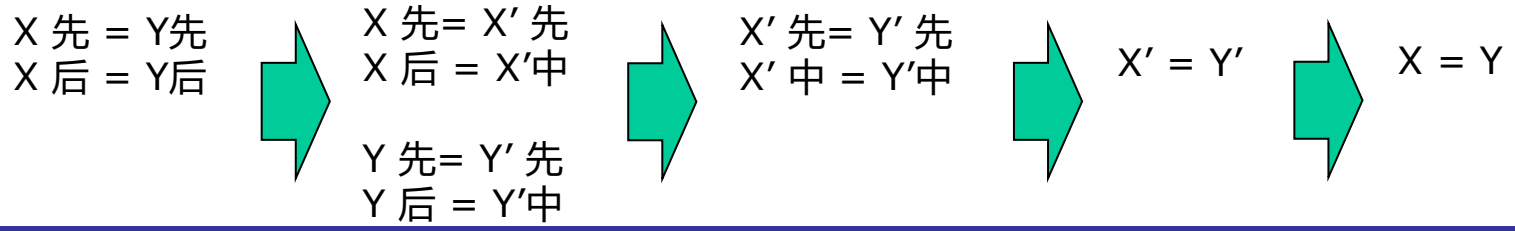
- 已知一棵树边的集合为{<A,B>, <A,C>, <B,D>, <B,E>, <B,F>, <C,G>, <C,H>, <E,I>, <E,J>}，在这棵树中，结点D的父结点是_____，结点E的层数是_____，结点B的度数为_____，后根深度优先遍历该树的序列为_____。

练习

- 结点最少的树是_____，结点最少的二叉树是_____。
- 设森林F中有三棵树，第一，第二，第三棵树的结点个数分别为M1，M2和M3。与森林F对应的二叉树根结点的右子树上的结点个数是_____。

  A．M1 B．M1+M2 C．M3 D．M2+M3

- 已知一棵树的先根次序遍历的结果与其对应二叉树表示(长子-兄弟表示)的前序遍历结果相同, 树的后根次序遍历结果与其对应二叉树表示的中序遍历结果相同。试问利用树的先根次序遍历结果和后根次序遍历结果能否唯一确定一棵树?如果正确，请证明，如果错误，举例说明。



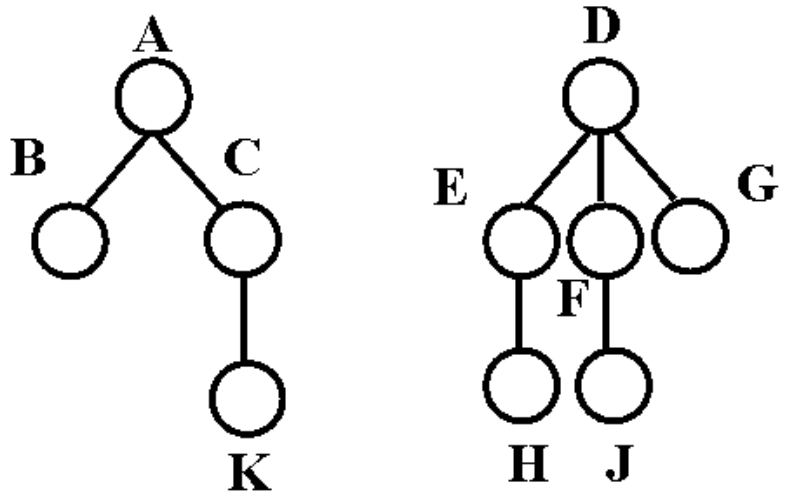
判断

- 二叉树的先根遍历序列中，任意一个结点均处在其子女结点的前面。
- 由树转换成二叉树，其根结点的右子树总是空的。
- 哈夫曼树是带权路径长度最短的树，路径上权值较大的结点离根较近。

思考题

- 带度数的先根次序？ Yes 利用栈实现
- 带度数的层次次序？ Yes 利用队列实现

树(森林)的图示



后根

degree	0	0	1	2	0	1	0	1	0	3
info	B	K	C	A	H	E	J	F	G	D

先根

A	B	C	K	D	E	H	F	J	G
2	0	1	0	3	1	0	1	0	0

层次

Root A	D	B	C	E	F	G	K	H	J	
2	2	3	0	1	1	1	0	0	0	0